

HEAPS

GOALS

- To explore the implementation, testing and performance of heap sort algorithm



HEAP

- A heap is a data structure that stores a collection of objects (with keys), and has the following properties:
 - Complete Binary tree
 - Heap Order
- It is implemented as an array where each node in the tree corresponds to an element of the array.
- An array is viewed as a nearly complete binary tree
 - **Physically** – linear array.
 - **Logically** – binary tree, filled on all levels (except lowest.)



HEAPS

A heap is a certain kind of complete binary tree.

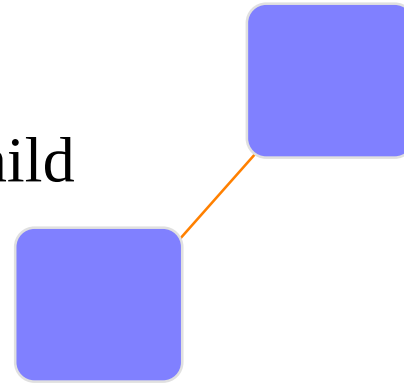
When a complete binary tree is built, its first node must be the root.



HEAPS

Complete binary tree.

Left child
of the
root

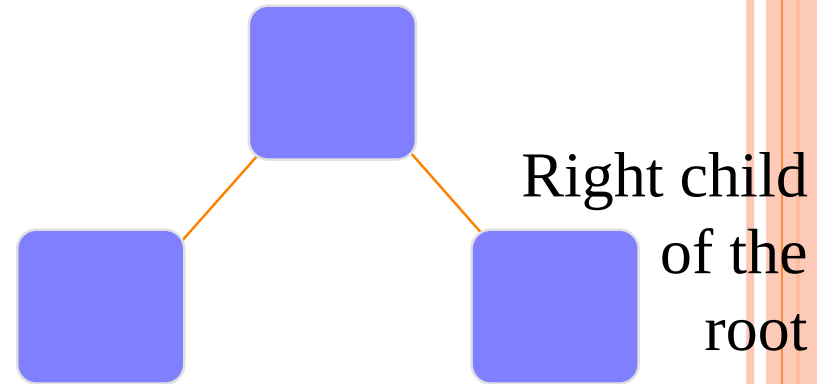


The second node is
always the left child
of the root.



HEAPS

Complete binary tree.

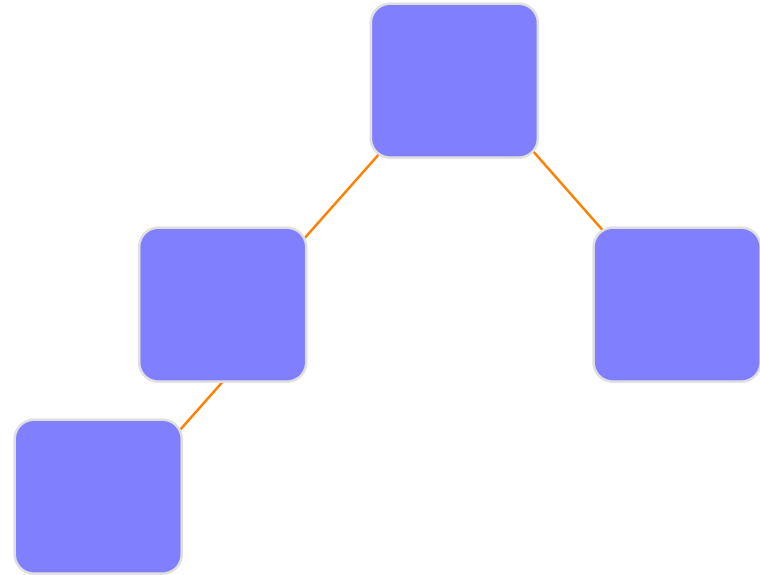


The third node is always the right child of the root.



HEAPS

Complete binary tree.

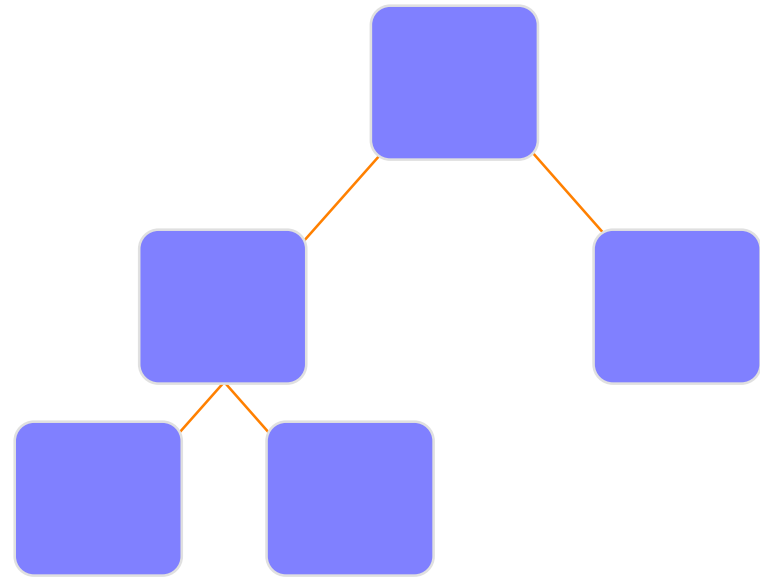


The next nodes
always fill the next
level from left-to-right.



HEAPS

Complete binary tree.

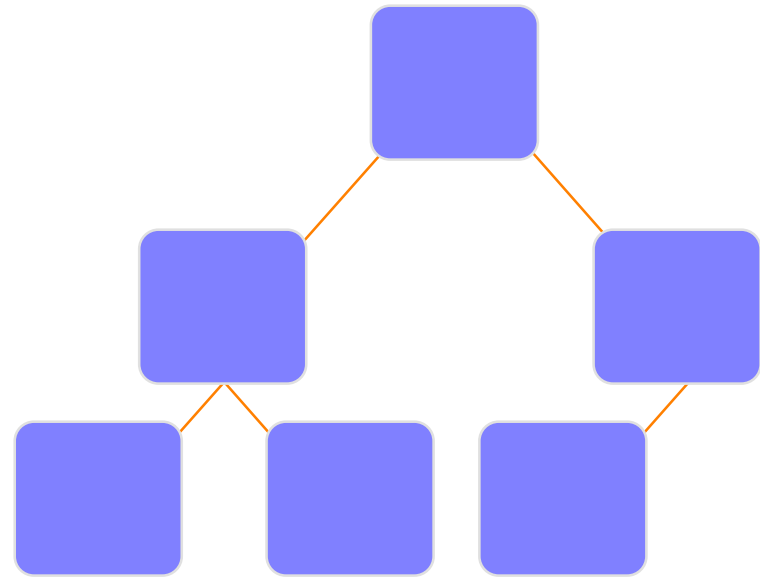


The next nodes
always fill the next
level from left-to-right.



HEAPS

Complete binary tree.

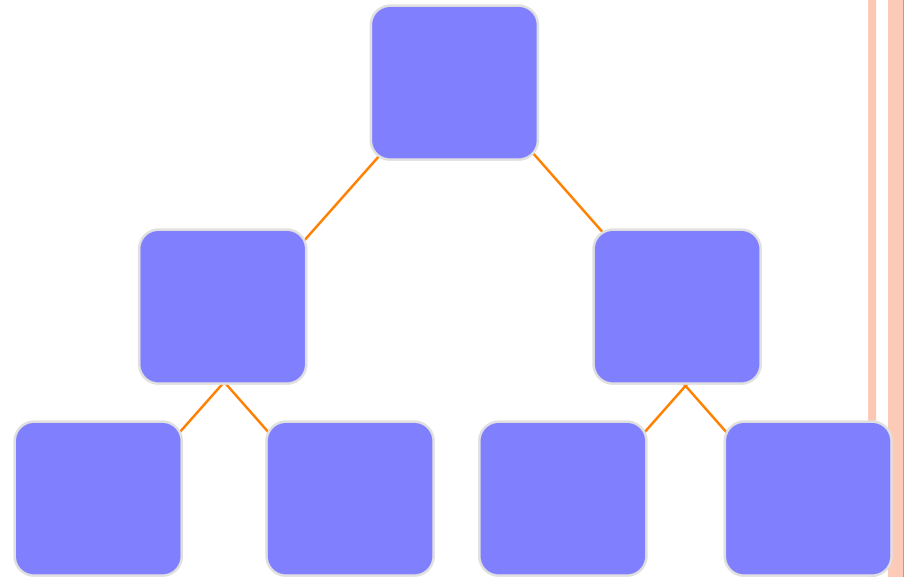


The next nodes
always fill the next
level from left-to-right.



HEAPS

Complete binary tree.

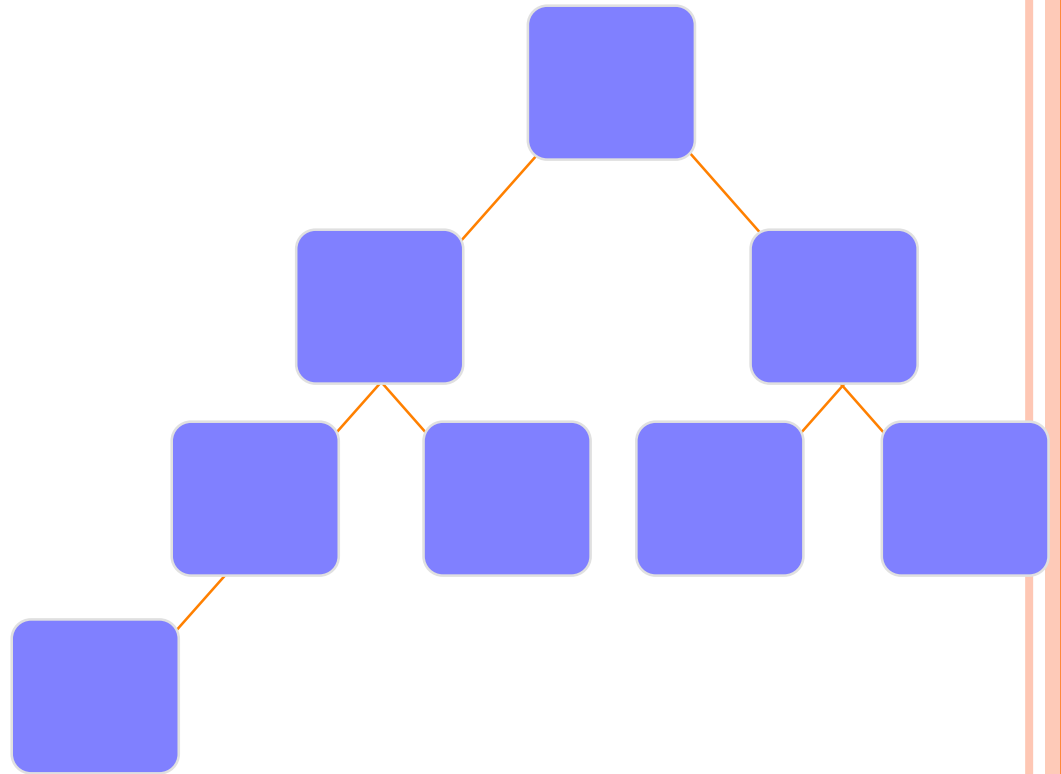


The next nodes
always fill the next
level from left-to-right.



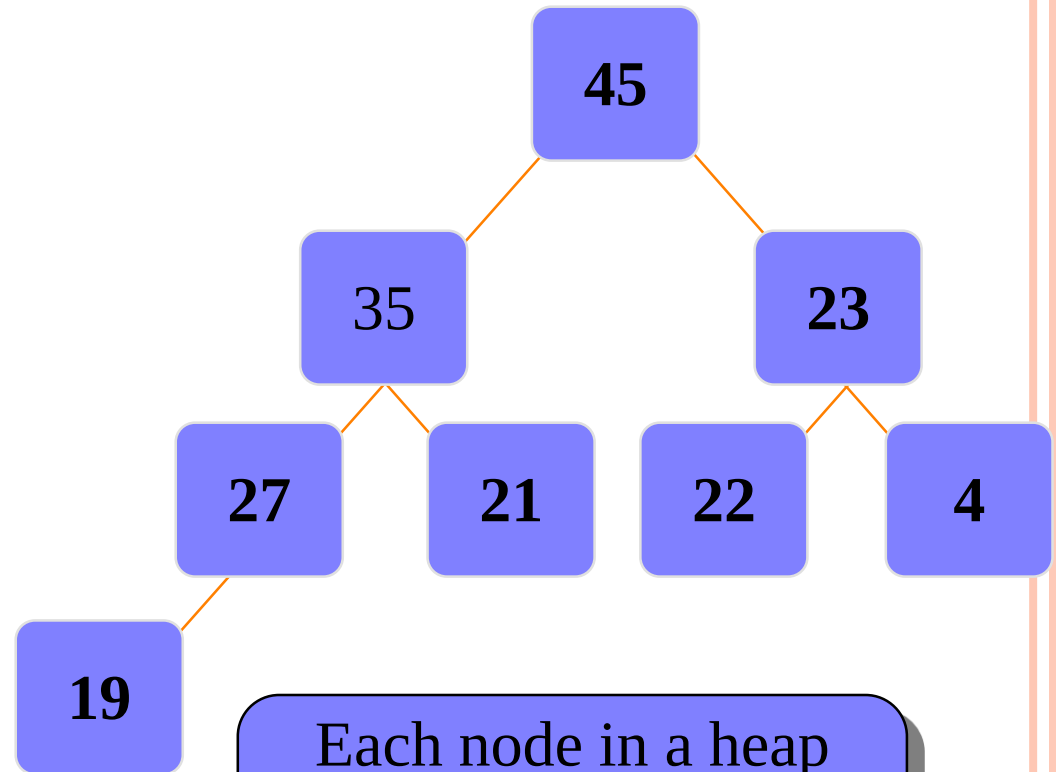
HEAPS

Complete binary tree.



HEAPS

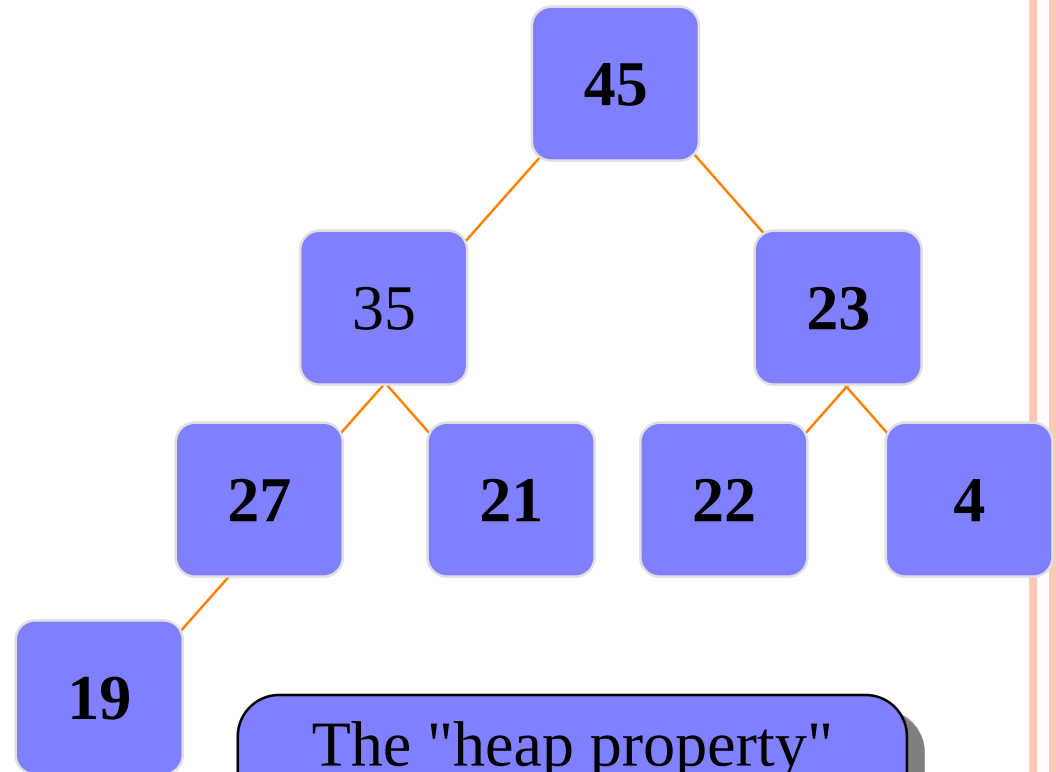
A heap is a certain kind of complete binary tree.



Each node in a heap contains a key that can be compared to other nodes' keys.

HEAPS

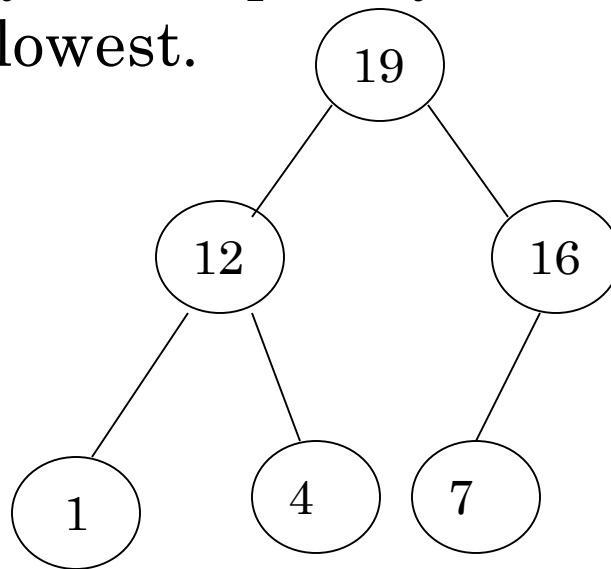
A heap is a certain kind of complete binary tree.



The "heap property" requires that each node's key is $\geq$ the keys of its children

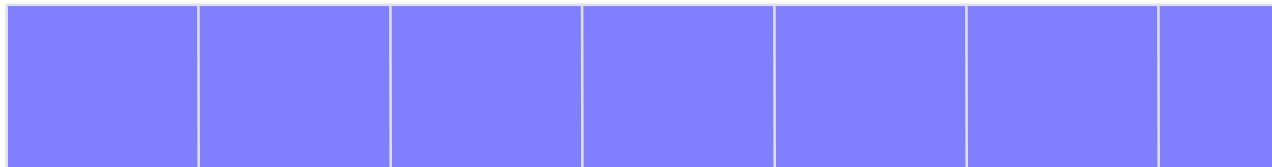
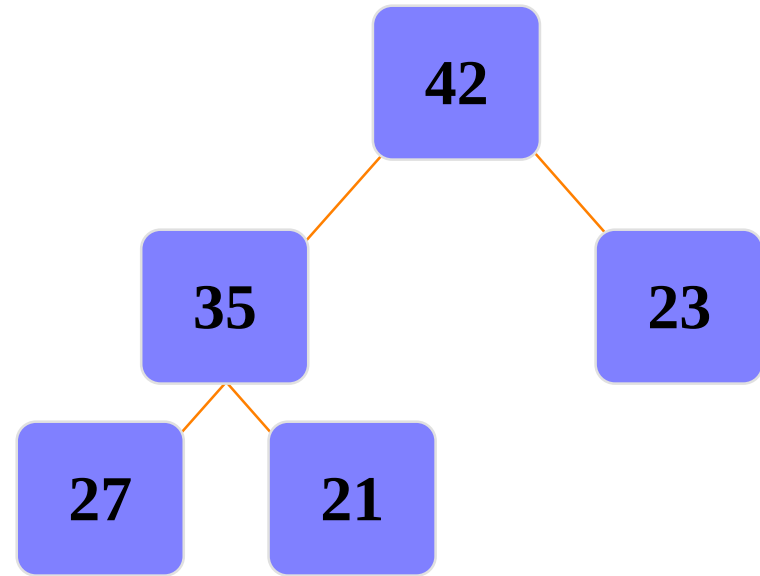
HEAP- DEFINITION

- The binary heap data structure is an array that can be viewed as a complete binary tree. Each node of the binary tree corresponds to an element of the array.
- The array is completely filled on all levels except possibly lowest.



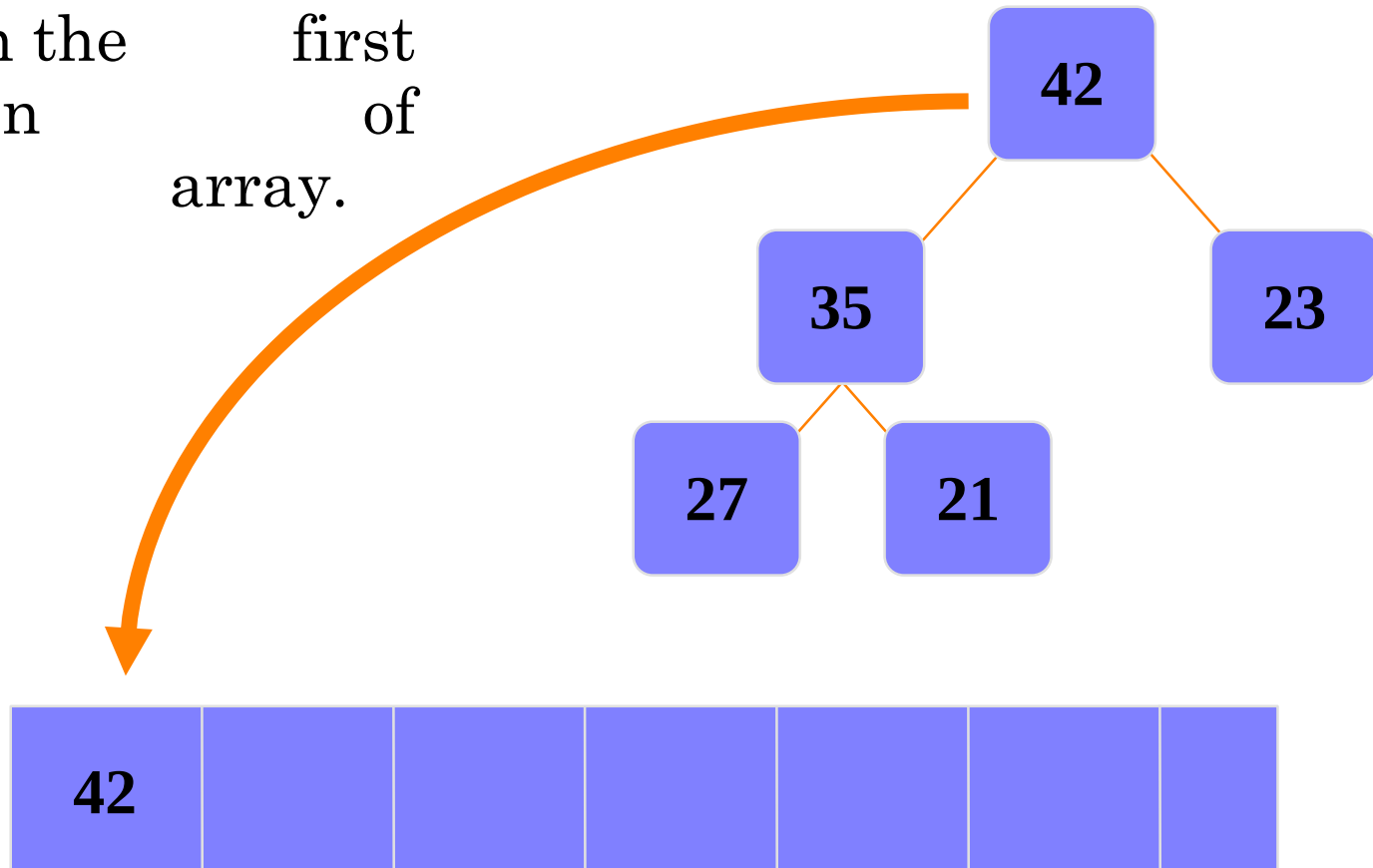
IMPLEMENTING A HEAP

- We will store the data from the nodes in a partially-filled array.



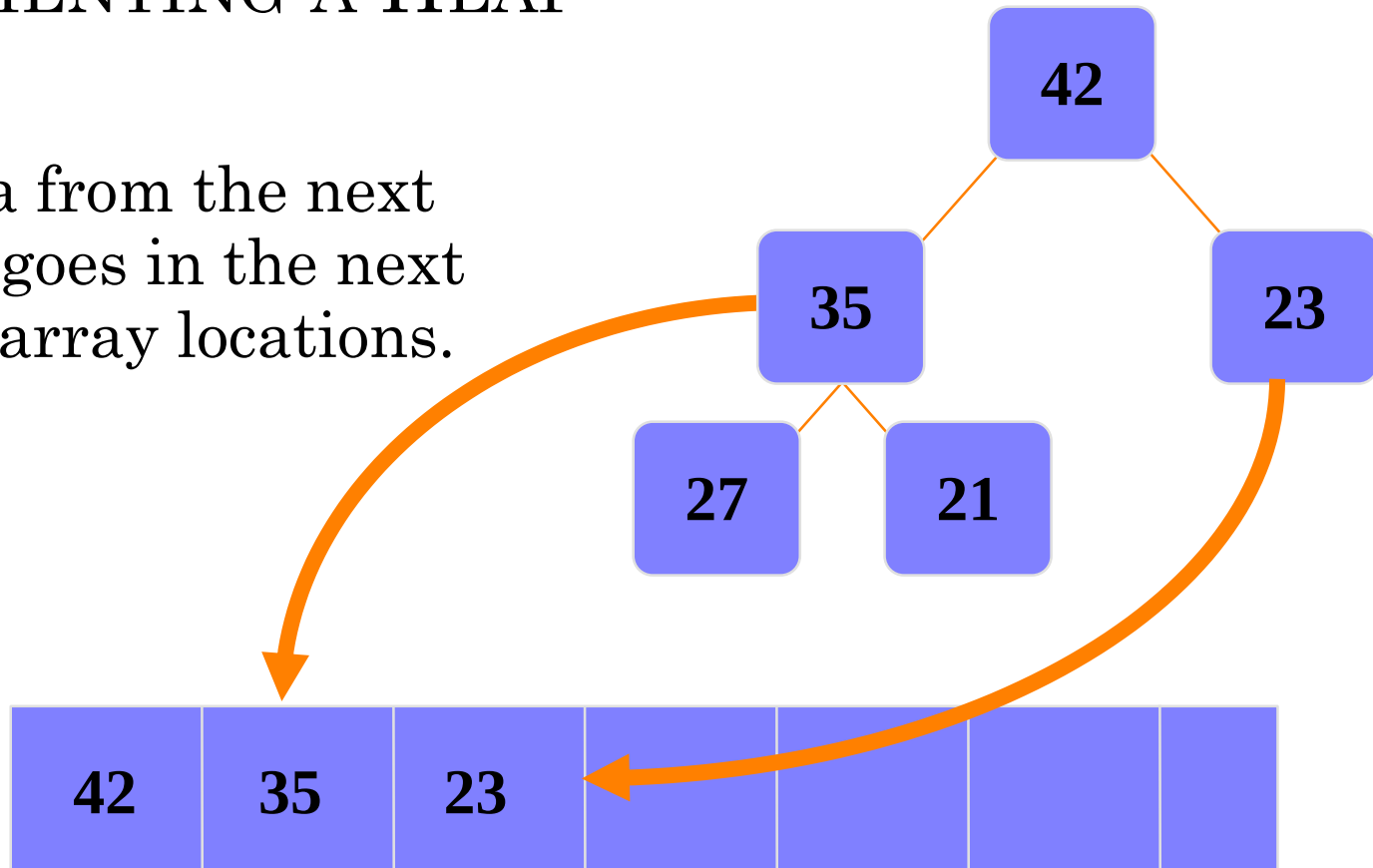
IMPLEMENTING A HEAP

- Data from the root goes in the first location of the array.



IMPLEMENTING A HEAP

- Data from the next row goes in the next two array locations.

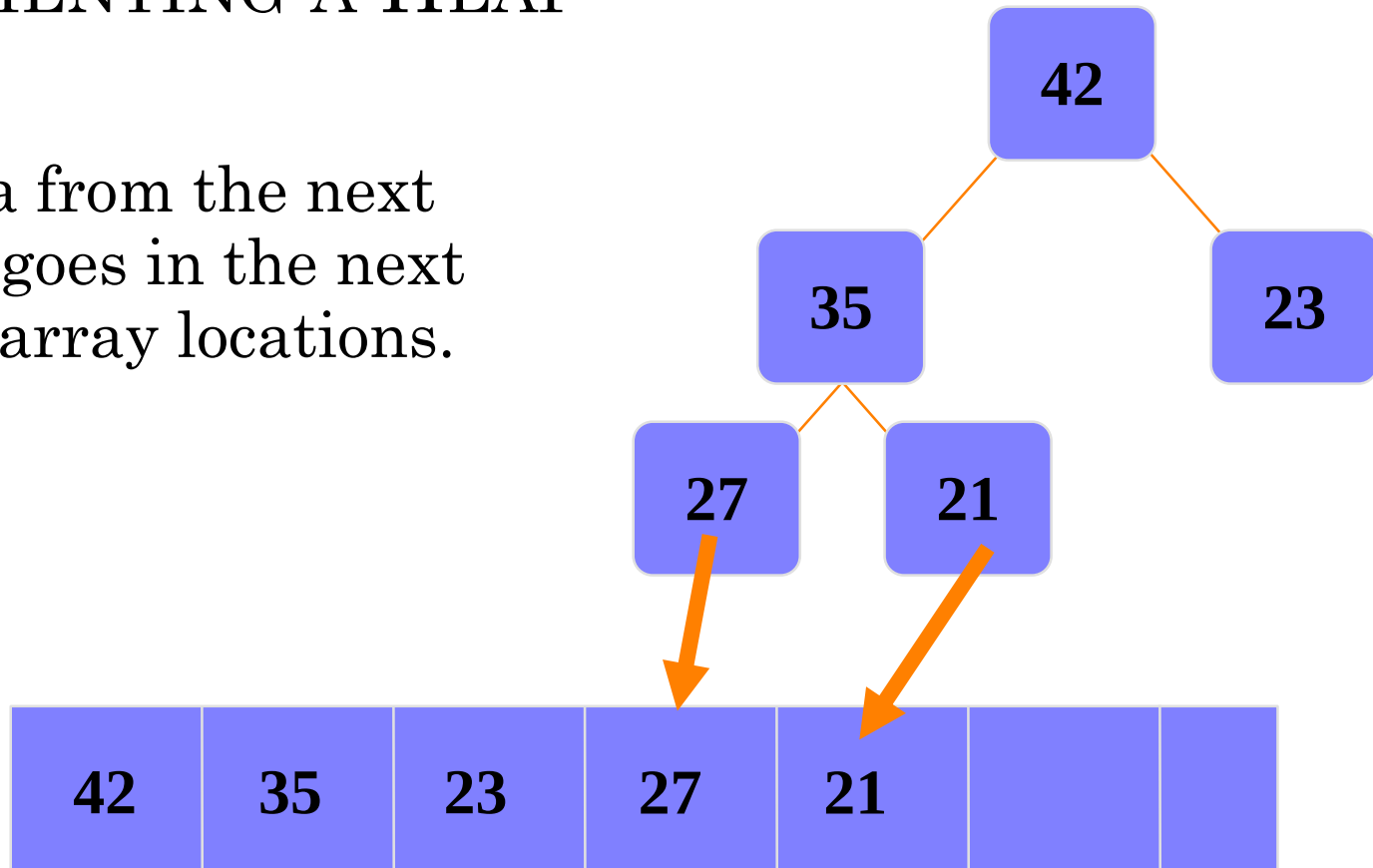


An array of data



IMPLEMENTING A HEAP

- Data from the next row goes in the next two array locations.

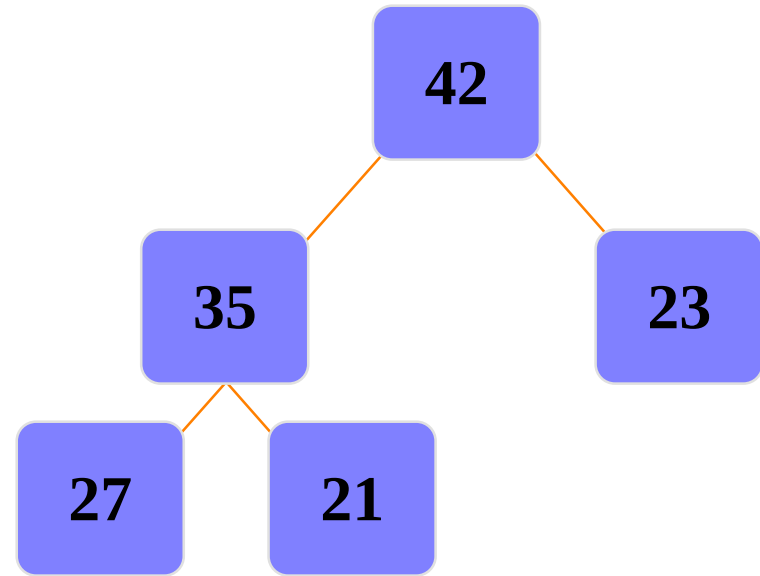


An array of data



IMPLEMENTING A HEAP

- Data from the next row goes in the next two array locations.

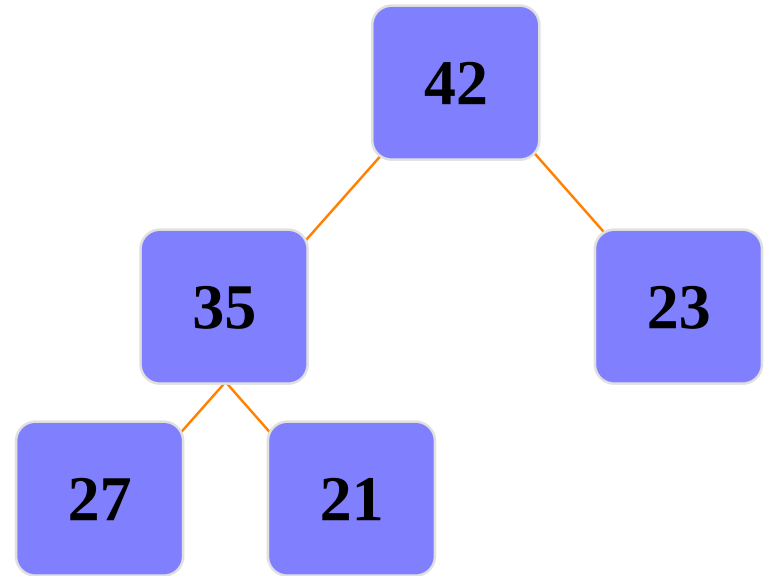


An array of data

We don't care what's in
this part of the array.

IMPORTANT POINTS ABOUT THE IMPLEMENTATION

- ❑ The links between the tree's nodes are not actually stored as pointers, or in any other way.
- ❑ The only way we "know" that "the array is a tree" is from the way we manipulate the data.

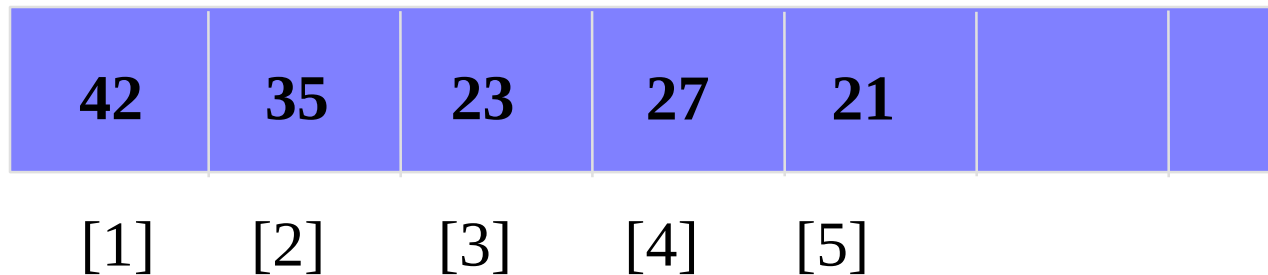
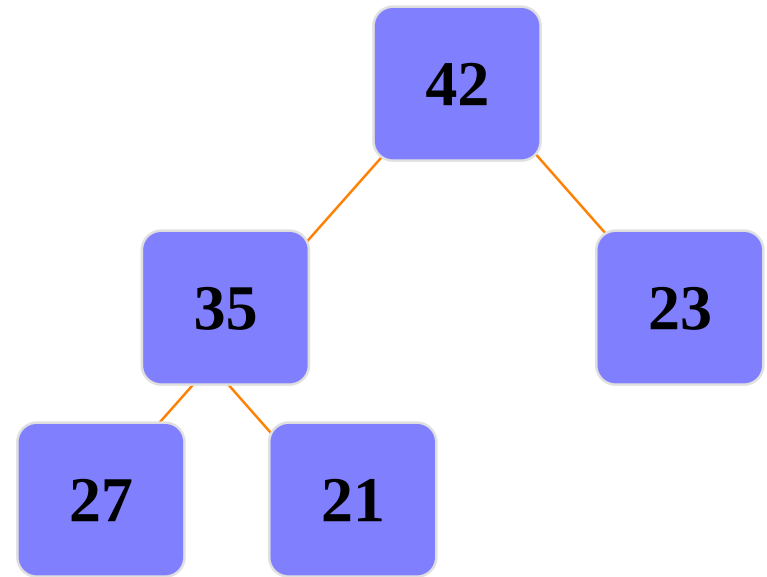


An array of data



IMPORTANT POINTS ABOUT THE IMPLEMENTATION

- If you know the index of a node, then it is easy to figure out the indexes of that node's parent and children.



HEAP

- The root of the tree $A[1]$ and given index i of a node, the indices of its parent, left child and right child can be computed

PARENT (i)

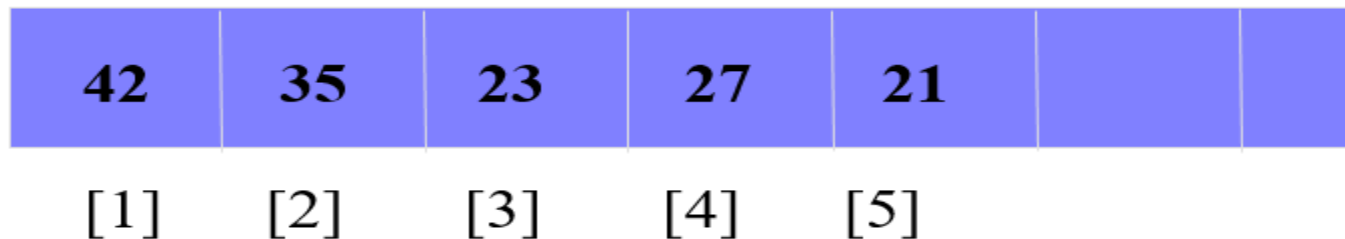
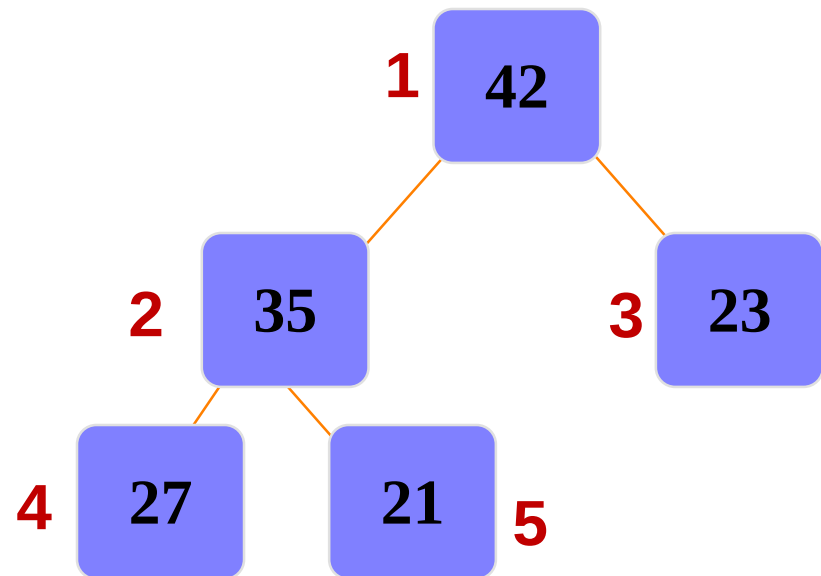
return $\text{floor}(i/2)$

LEFT (i)

return $2i$

RIGHT (i)

return $2i + 1$



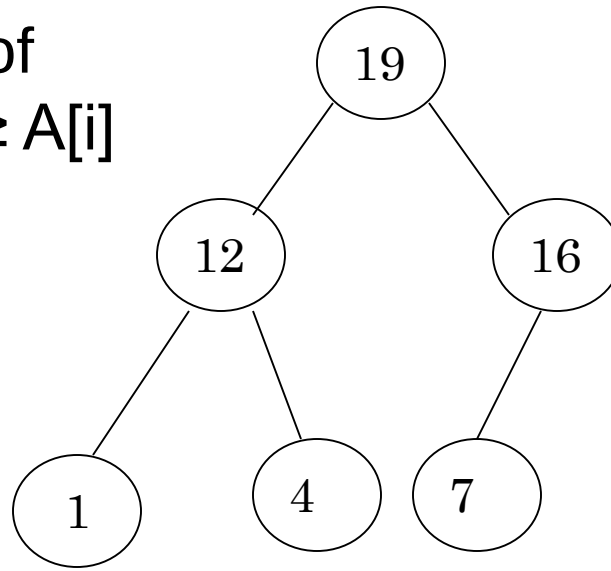
HEAP ORDER PROPERTY

- For every node v , other than the root, the key stored in v is greater or equal (smaller or equal for max heap) than the key stored in the parent of v .
- In this case the maximum value is stored in the root



MAX HEAP EXAMPLE

- Store data in ascending order
- Has property of $A[\text{Parent}(i)] \geq A[i]$



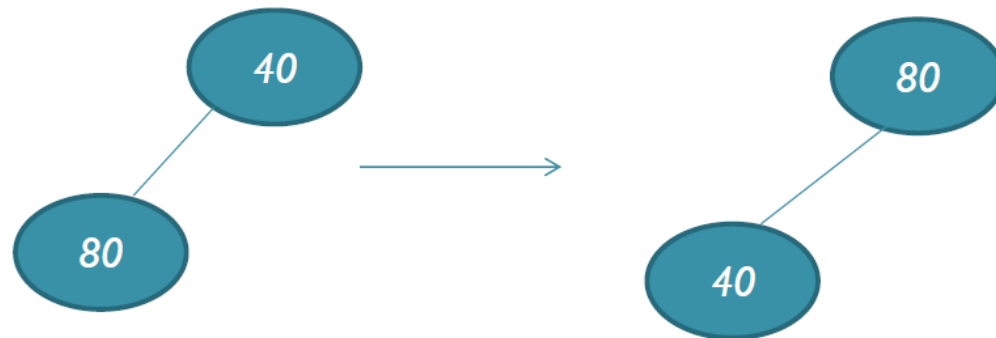
CONSTRUCTION OF MAX HEAP - EXAMPLE

- Input data = {40,80,35,90, 45,50,70}

- Insert 40

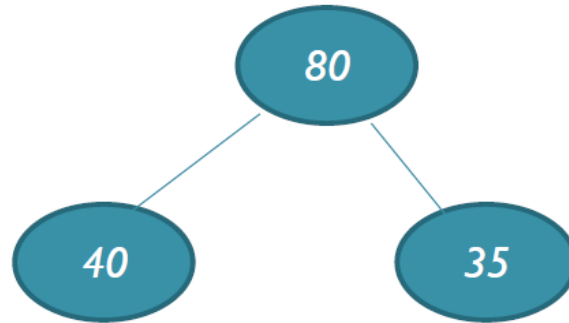


- Insert 80

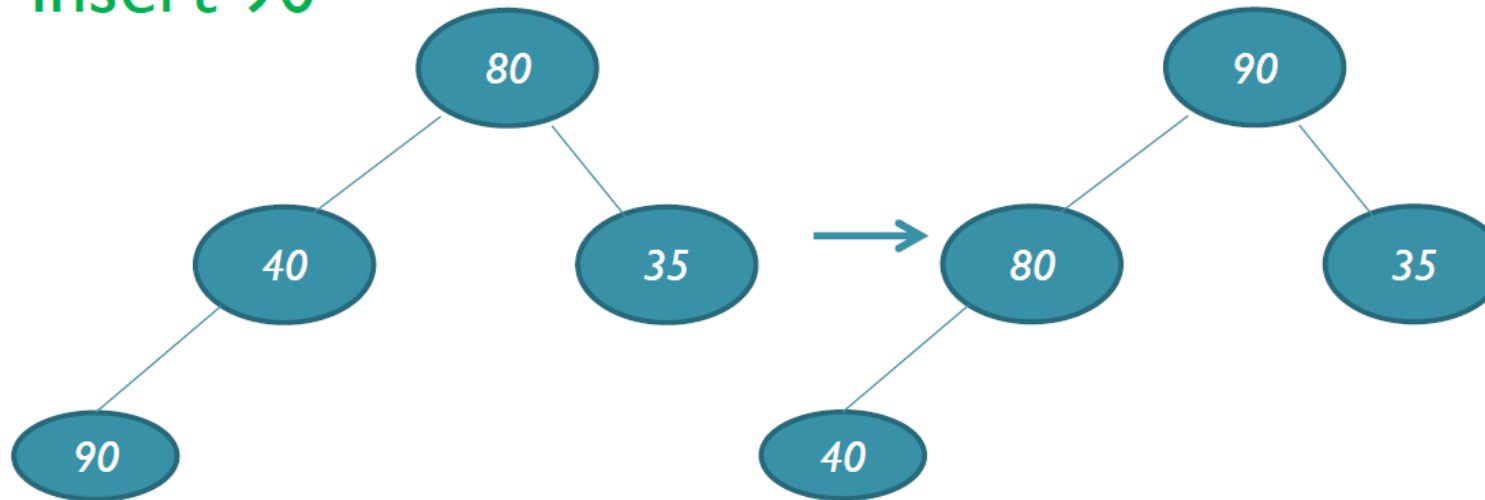


CONSTRUCTION OF MAX HEAP - EXAMPLE

- Insert 35

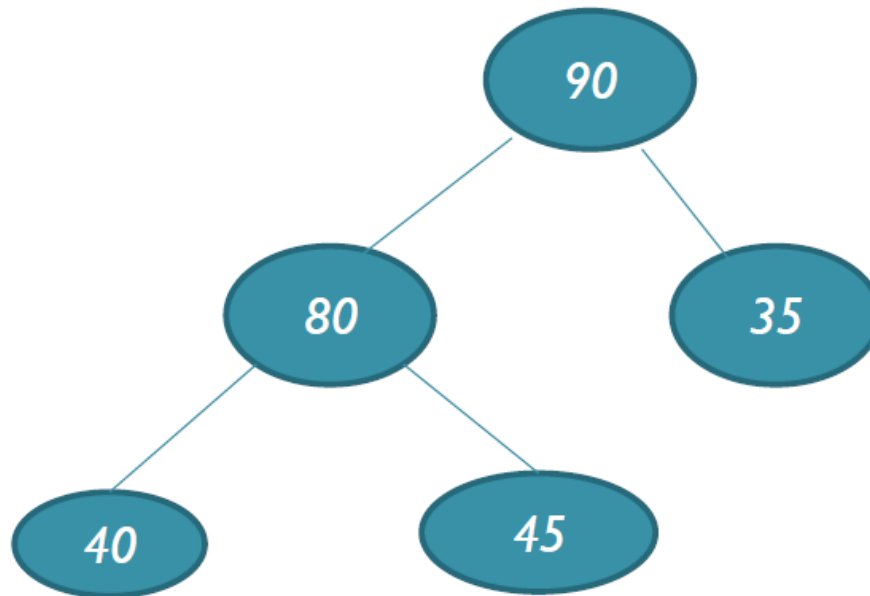


- Insert 90



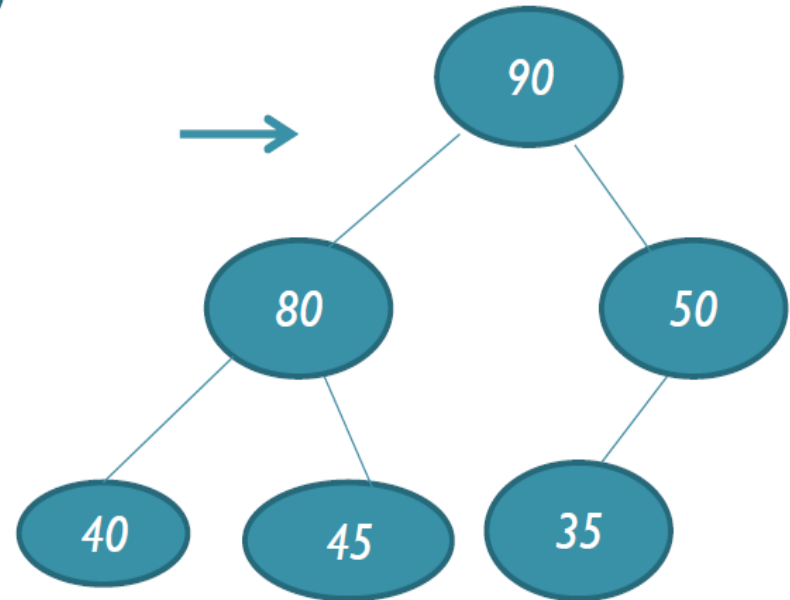
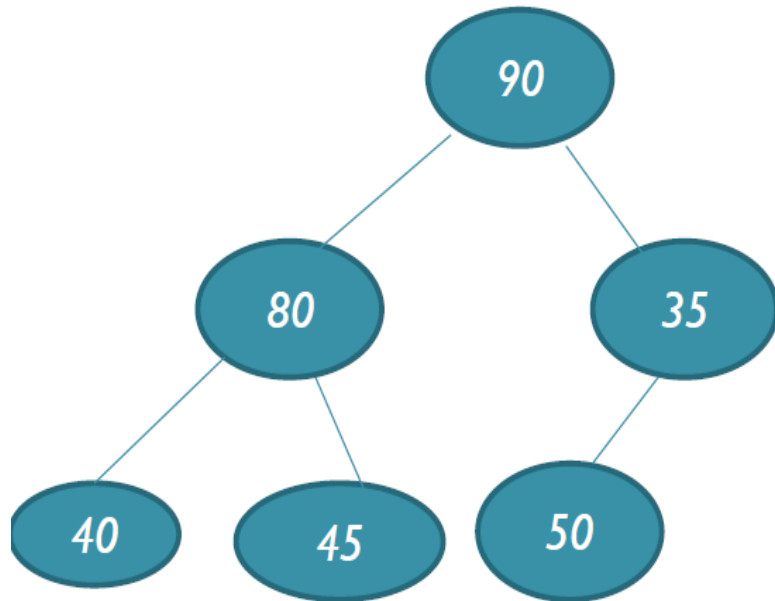
CONSTRUCTION OF MAX HEAP - EXAMPLE

- Insert 45

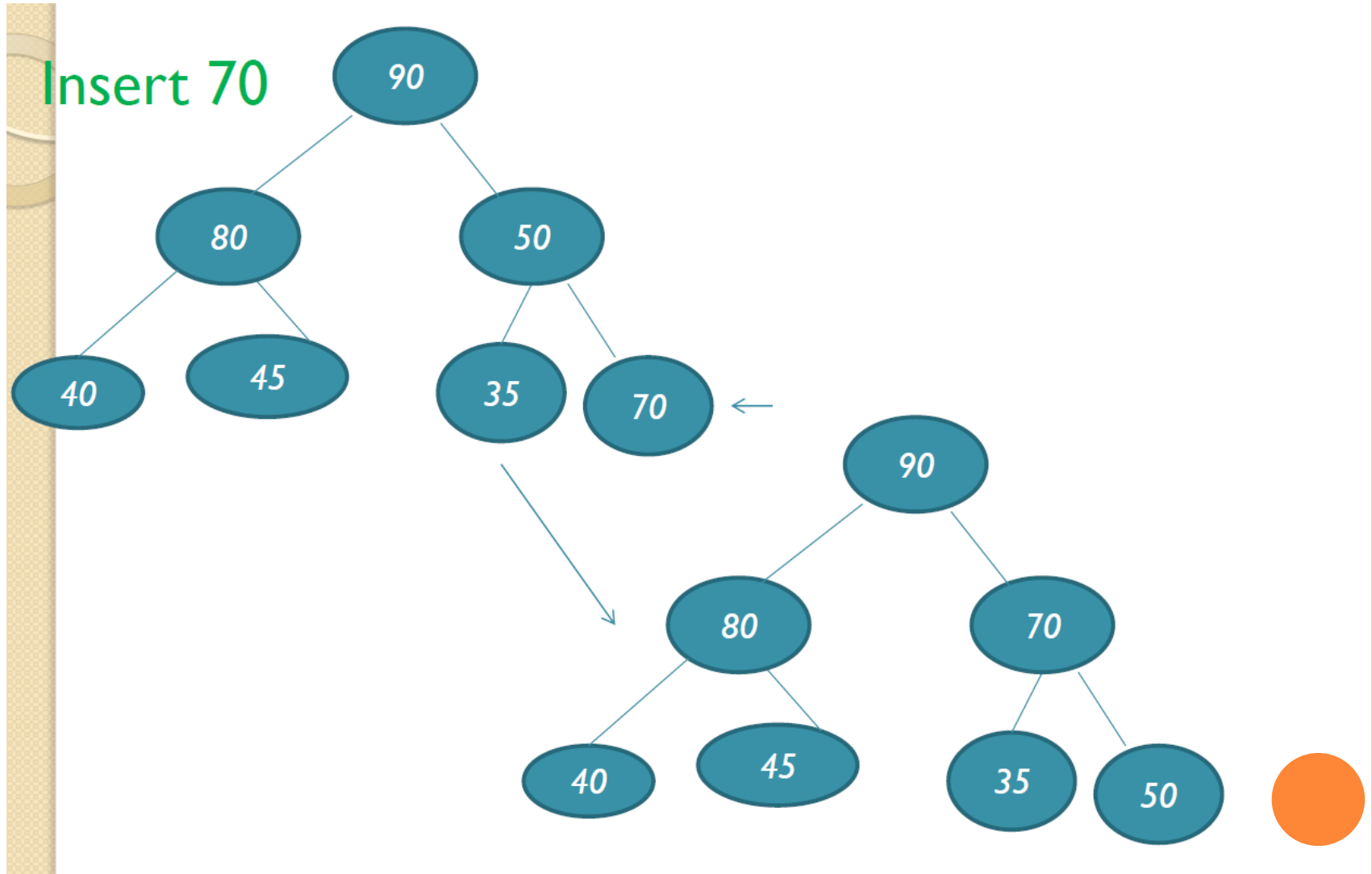


CONSTRUCTION OF MAX HEAP - EXAMPLE

- Insert 50



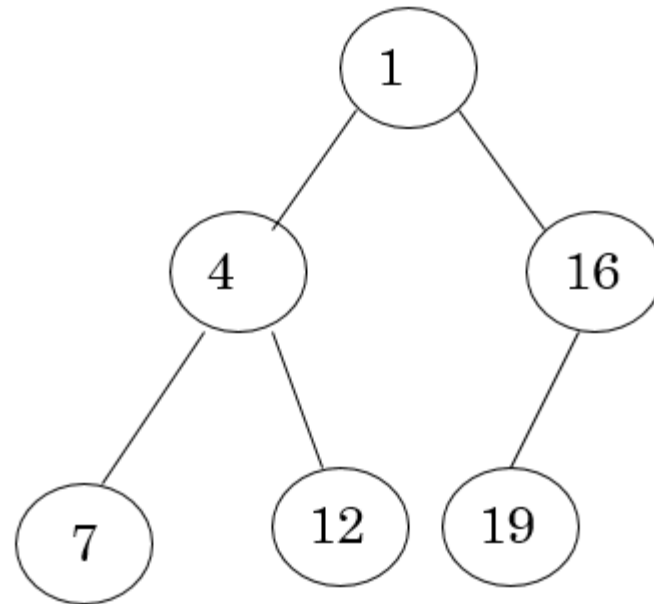
CONSTRUCTION OF MAX HEAP - EXAMPLE



DEFINITION

○ Min Heap

- Store data in descending order
- Has property of $A[\text{Parent}(i)] \leq A[i]$



INSERTION

- Algorithm

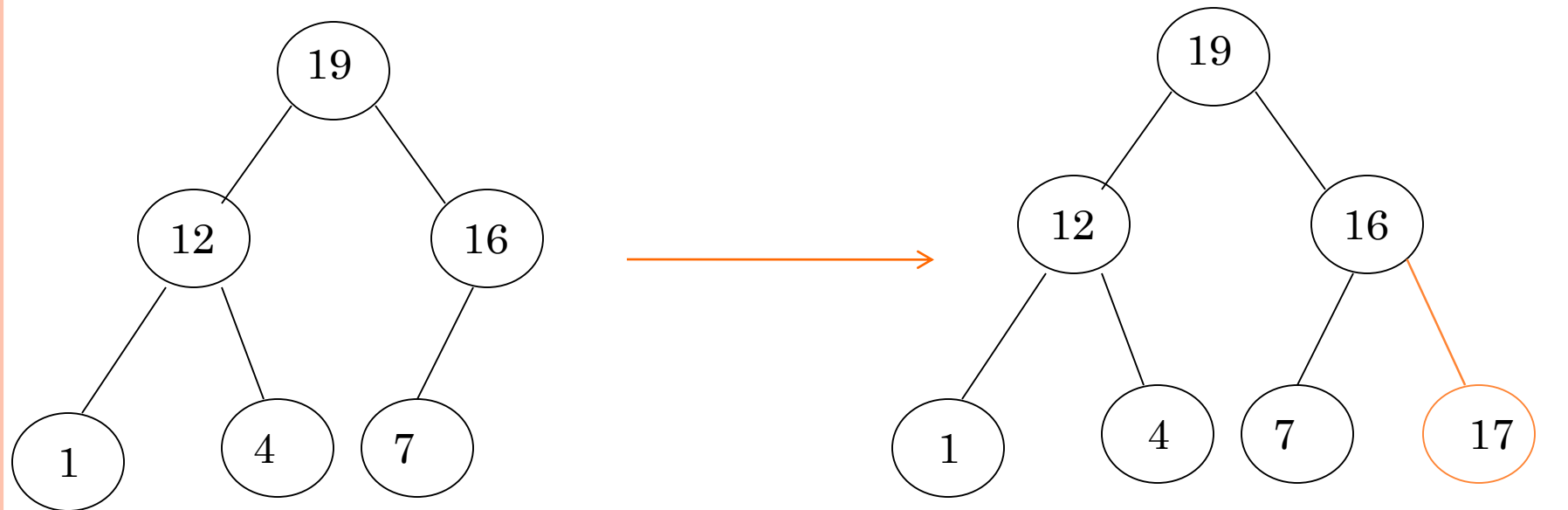
1. Add the new element to the next available position at the lowest level
2. Restore the max-heap property if violated
 - General strategy is percolate up (or bubble up): if the parent of the element is smaller than the element, then interchange the parent and child.

OR

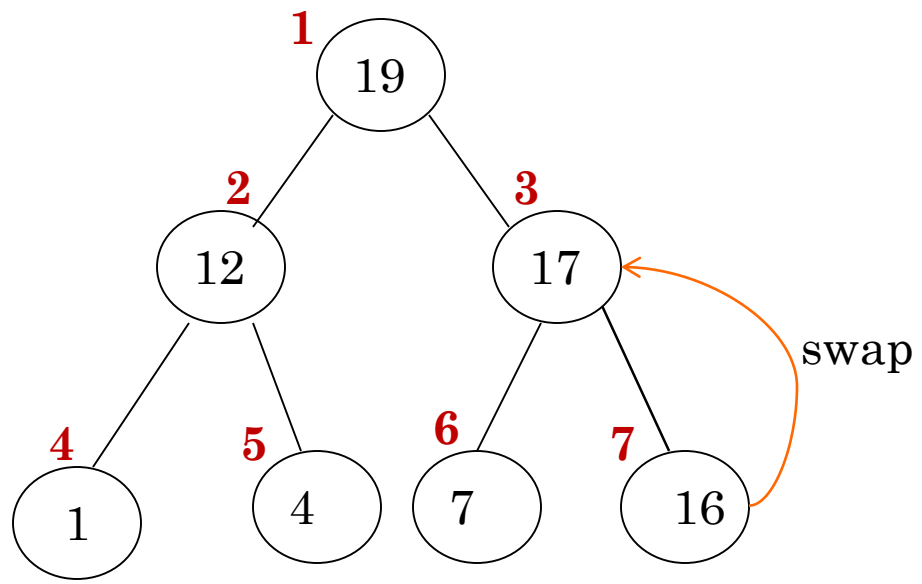
Restore the min-heap property if violated

- General strategy is percolate up (or bubble up): if the parent of the element is larger than the element, then interchange the parent and child.



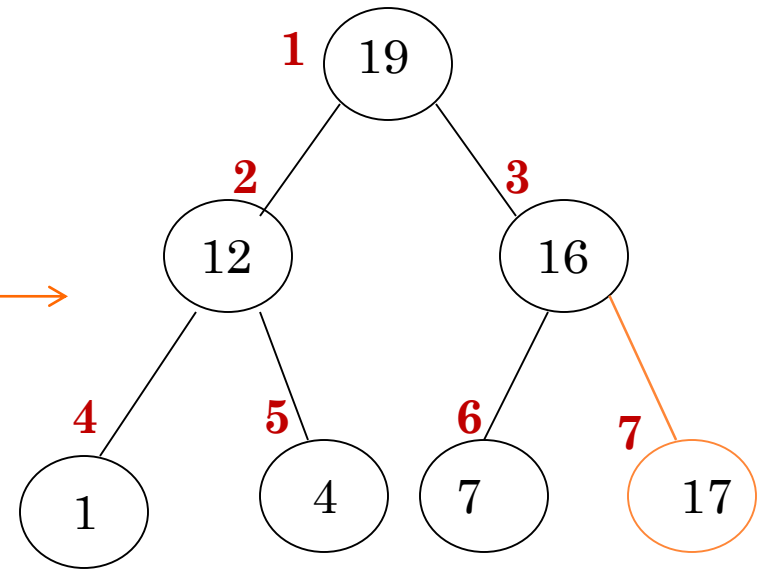
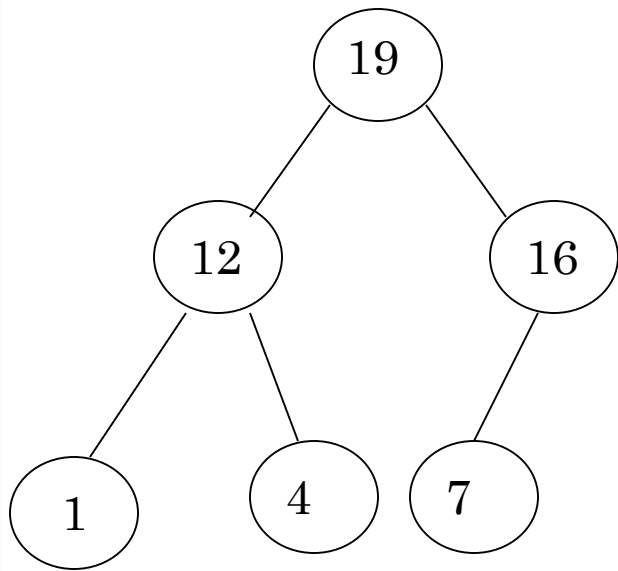


Insert 17



Percolate up to maintain the heap property





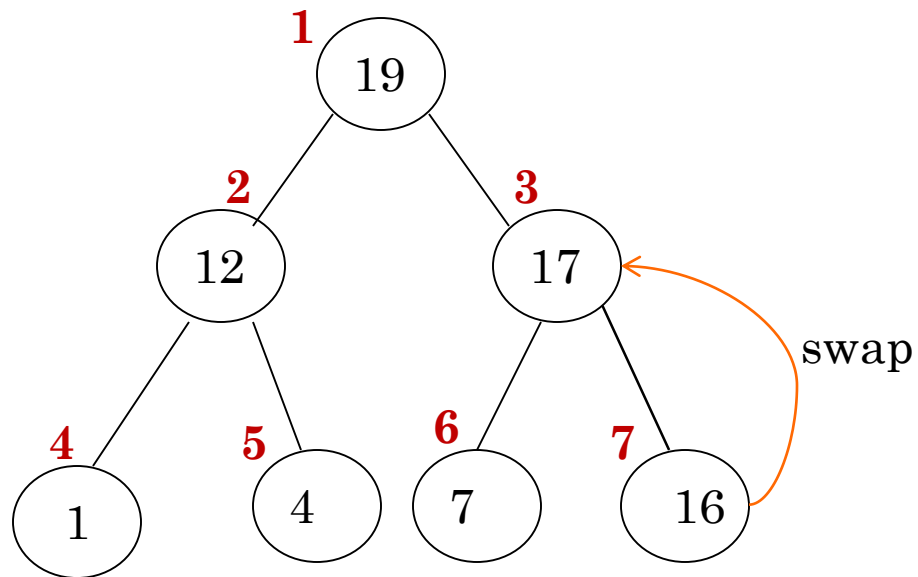
Insert 17

PARENT (i)
return floor($i/2$)

19	12	16	1	4	7	17
1	2	3	4	5	6	7



19	12	17	1	4	7	16
1	2	3	4	5	6	7



Percolate up to maintain the heap property

Data: B : input array; N : starting index; $newValue$: new node

Result: Heap tree with the new node

Procedure $Insertion(B, N, newValue)$ // Array B , heap size N , new node $newValue$

$N = N + 1$; // Create a space in a heap to add the new node

$B[N] = newValue$;

$k = N$;

while $k > 1$ **do**

$PNode = k / 2$; // $PNode$ to denote the parent node

if $B[PNode] < B[k]$ **then**

$swap(B[PNode], B[k])$;

$k = PNode$;

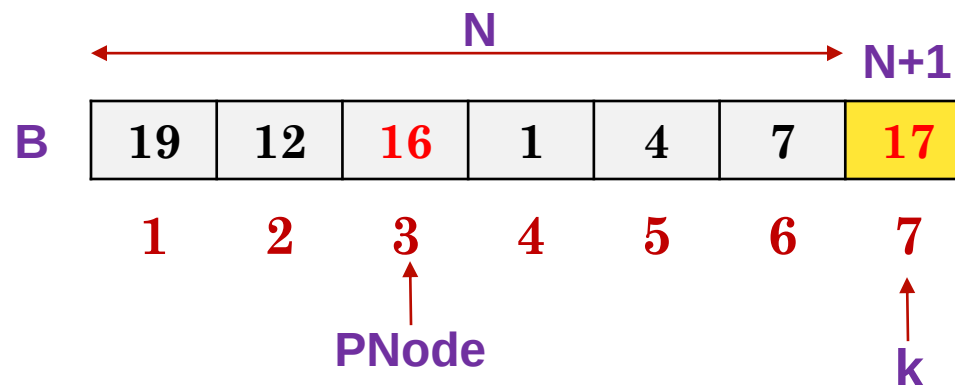
else

return;

end

end

end



DELETION

○ Delete max

- Copy the last number to the root (overwrite the maximum element stored there).
- Restore the max heap property by percolate down.

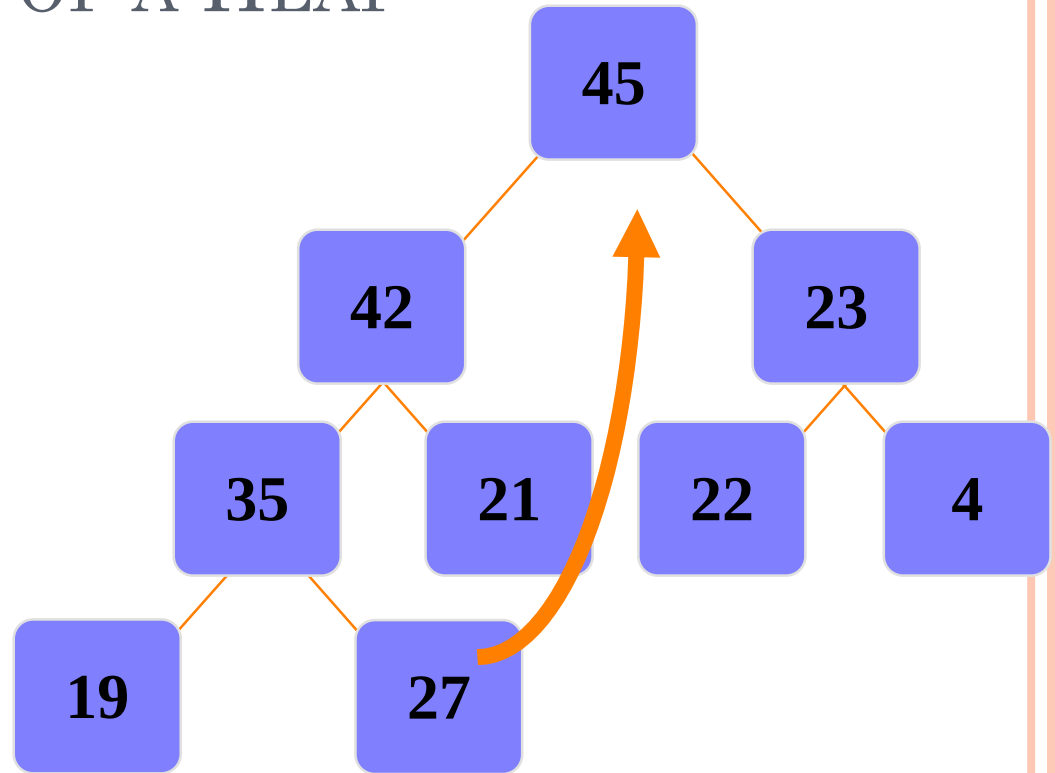
○ Delete min

- Copy the last number to the root (overwrite the minimum element stored there).
- Restore the min heap property by percolate down.



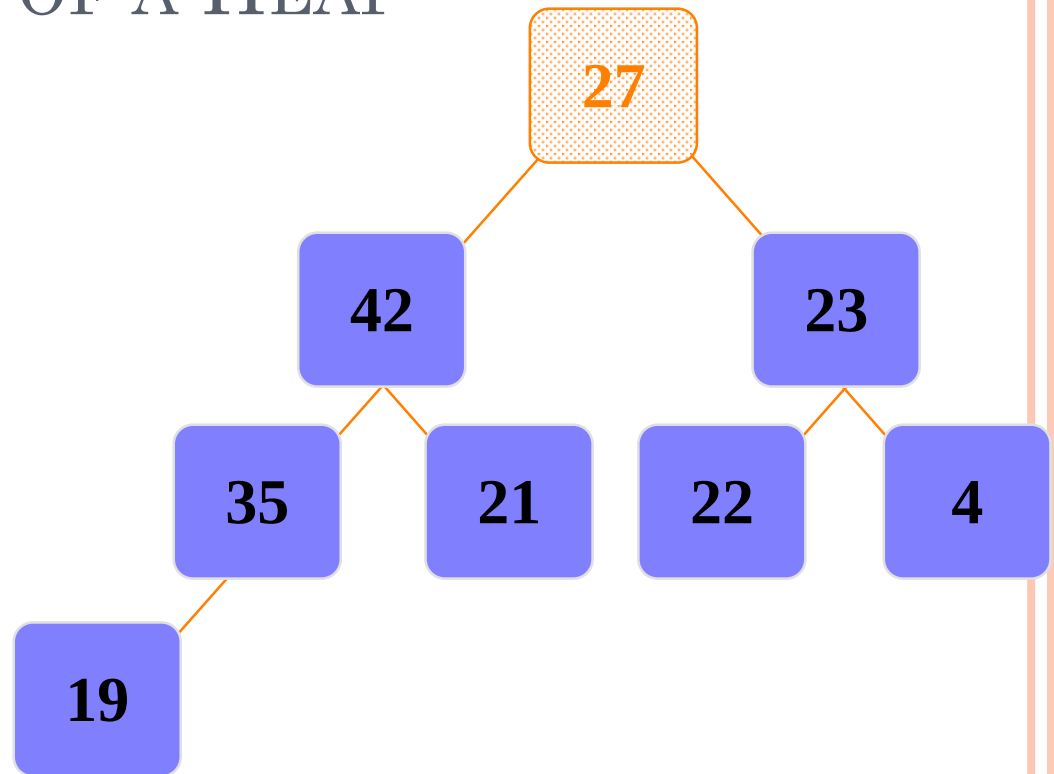
REMOVING THE TOP OF A HEAP

- Move the last node onto the root.



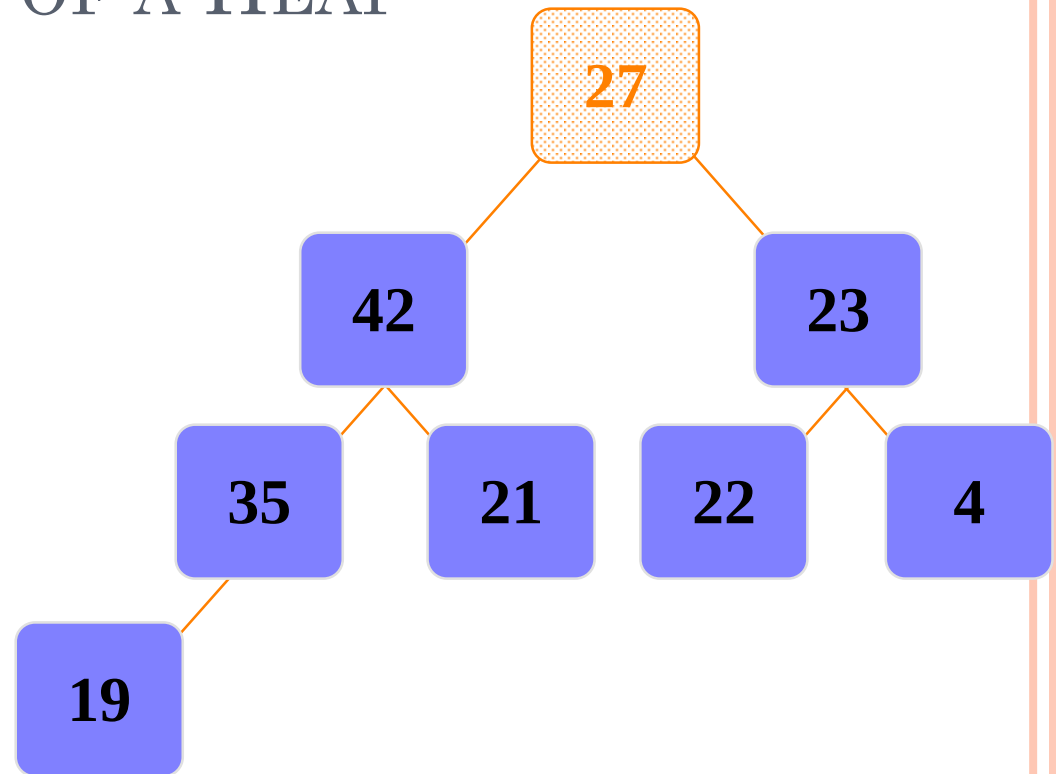
REMOVING THE TOP OF A HEAP

- Move the last node onto the root.



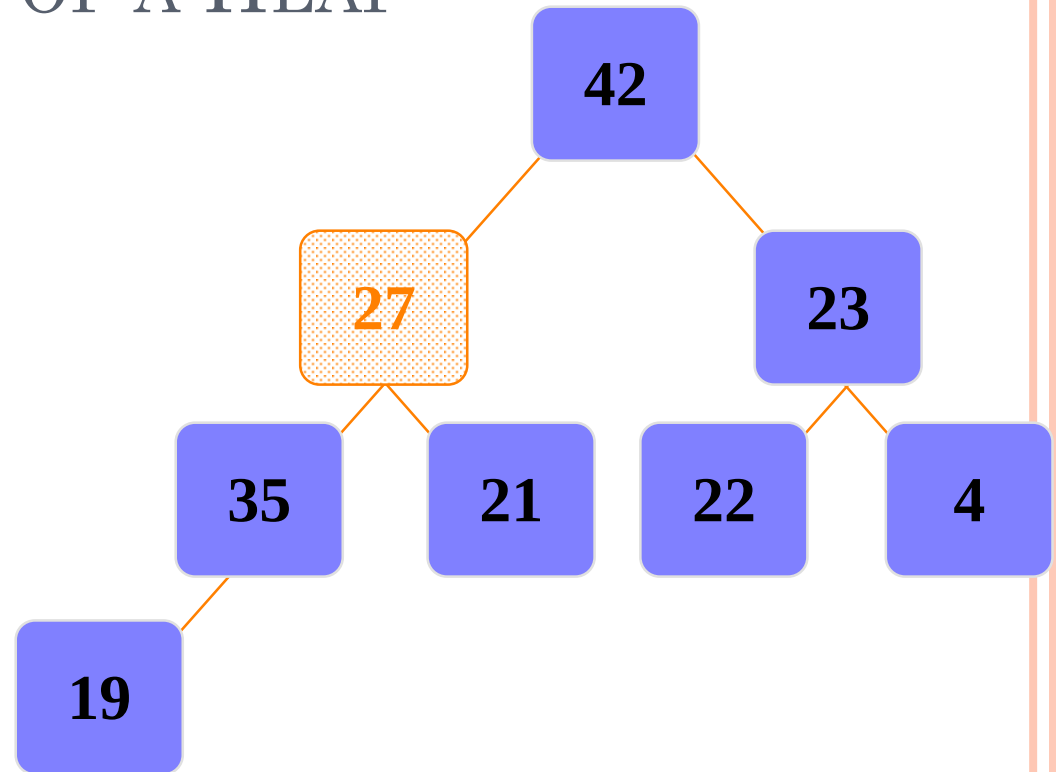
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



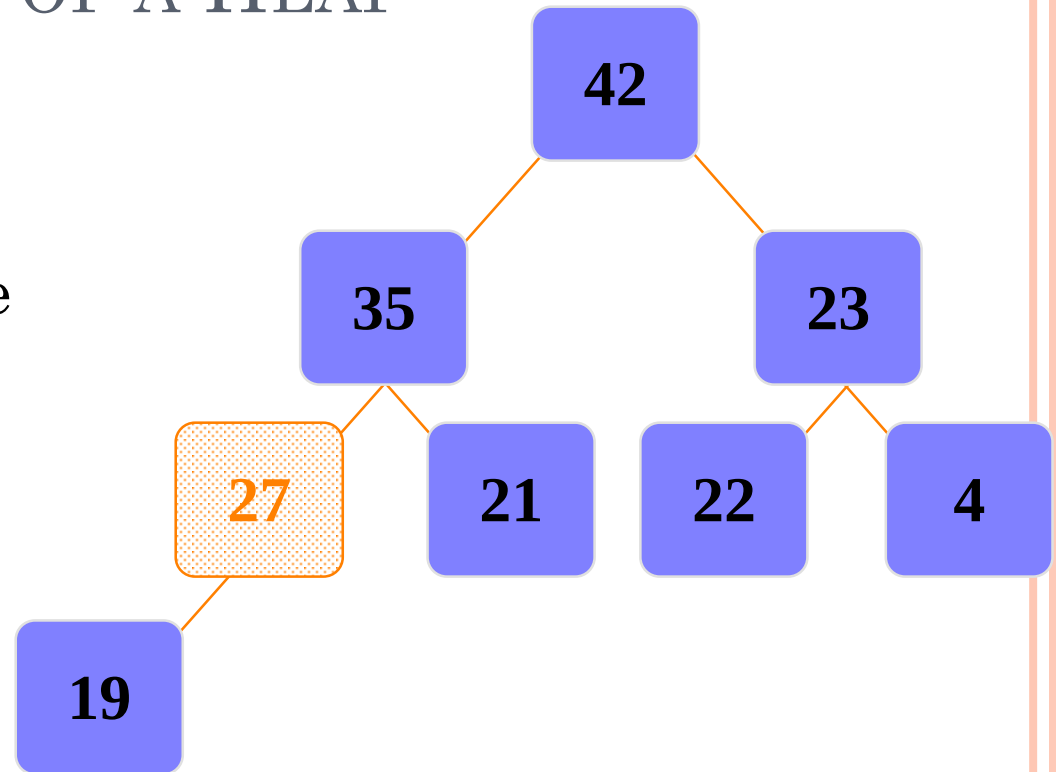
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



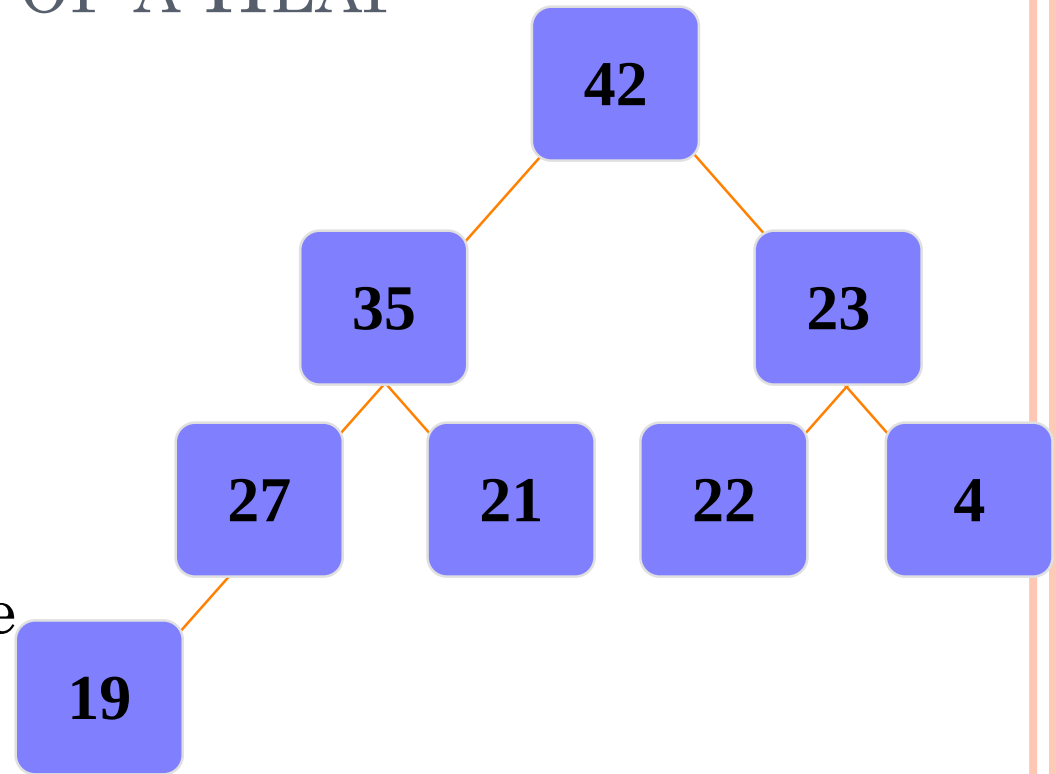
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.

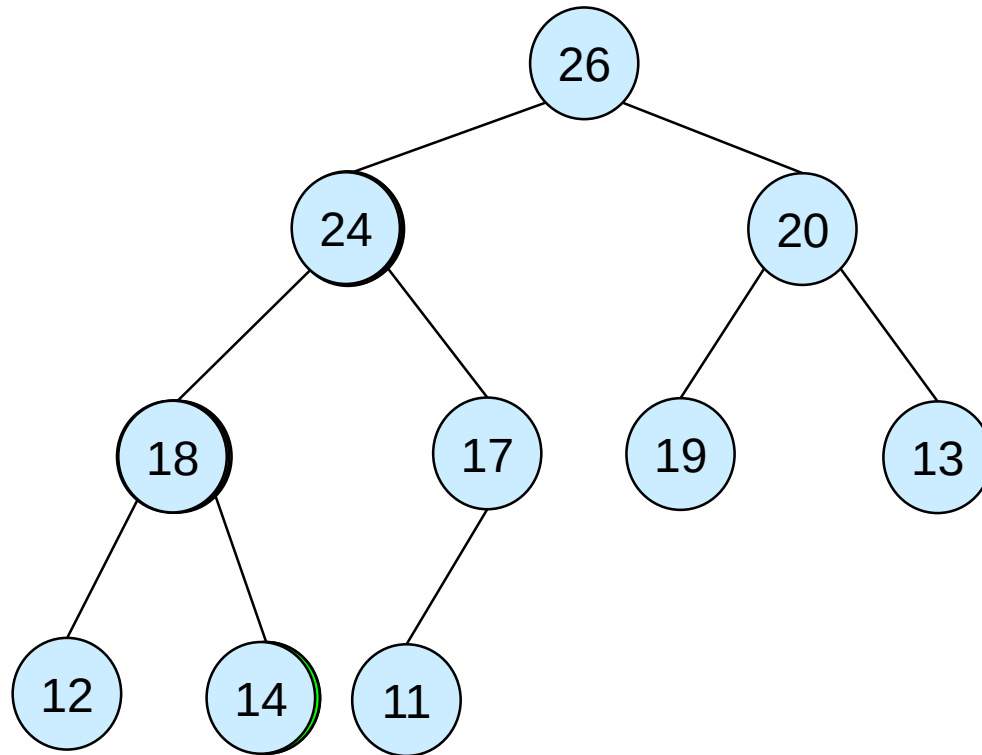


REMOVING THE TOP OF A HEAP

- ❑ The children all have keys $\leq$ the out-of-place node, or
- ❑ The node reaches the leaf.
- ❑ The process of pushing the new node downward is called reheapification downward.



MAXHEAPIFY – EXAMPLE



HEAP SORT

- Heapsort is a popular and efficient sorting algorithm.
- The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.



PROCEDURES ON HEAP SORT

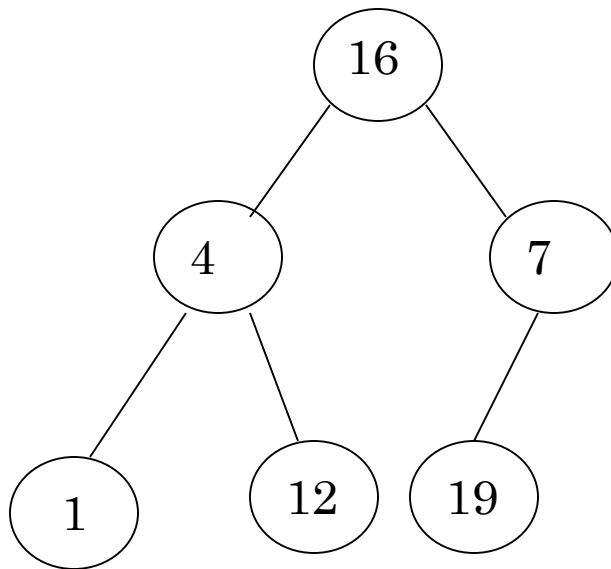
- In heap sorting, basically, there are two phases involved in the sorting of elements.
- By using the heap sort algorithm, they are as follows -
 - The first step includes the creation of a heap by adjusting the elements of the array.
 - After the heap creation, remove the heap's root element repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

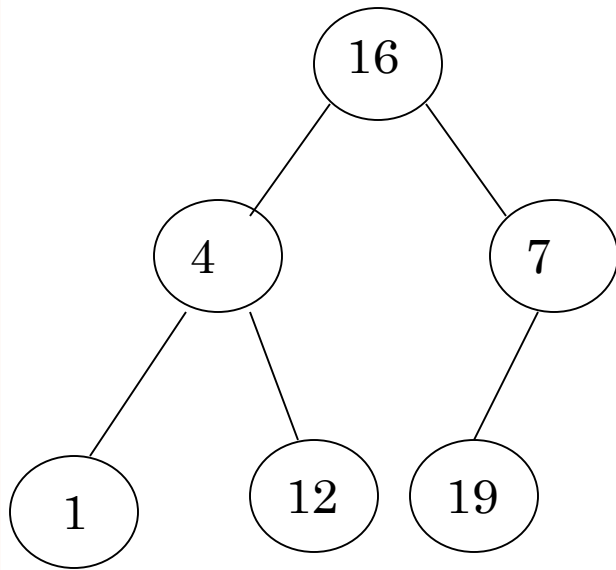


Example: Convert the following array to a heap

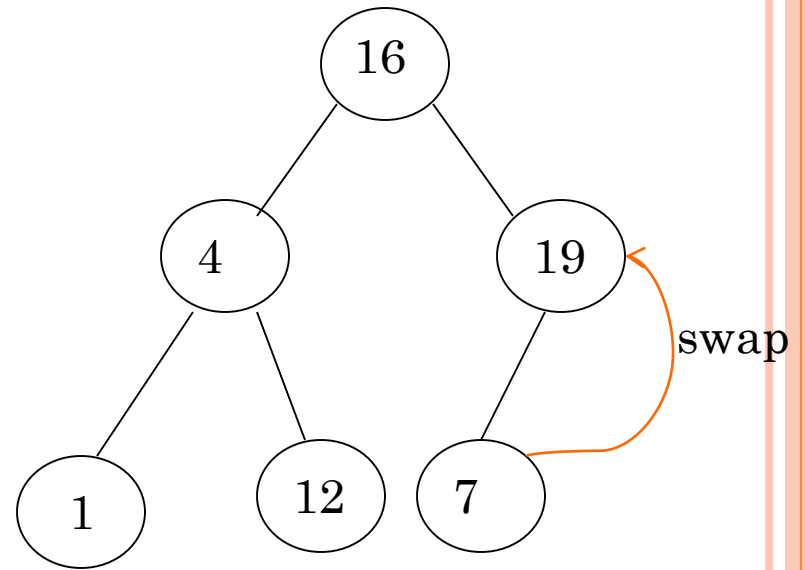
16	4	7	1	12	19
----	---	---	---	----	----

Picture **the array as a complete binary tree:**

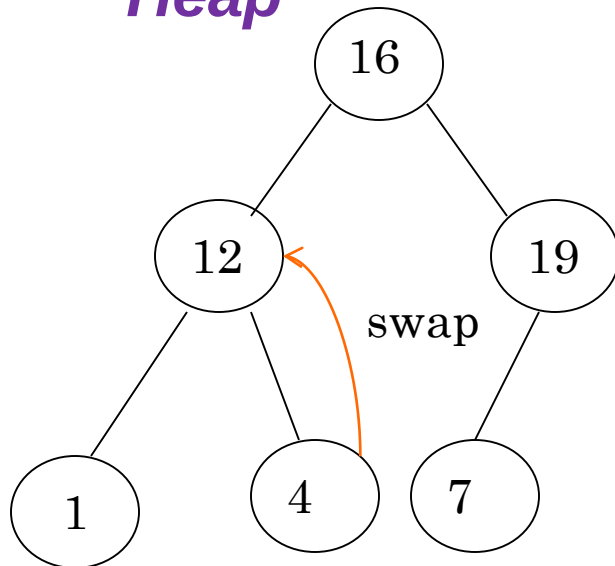




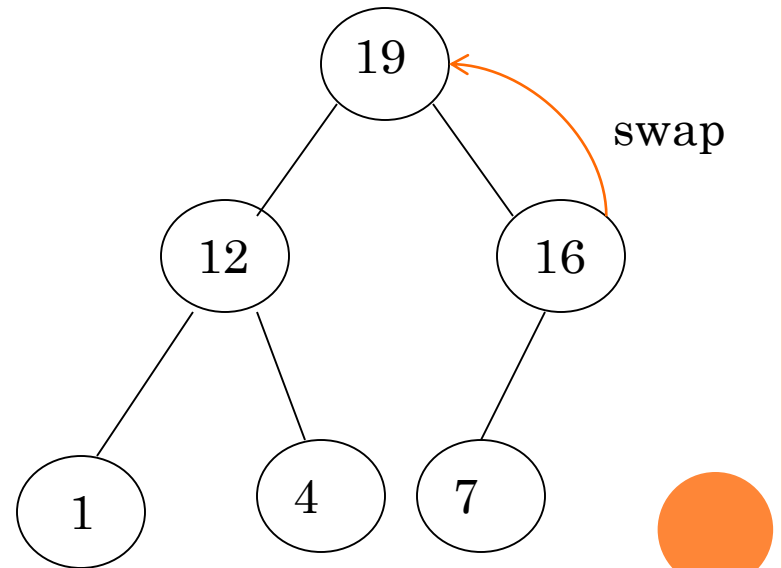
Heapify →



Heap



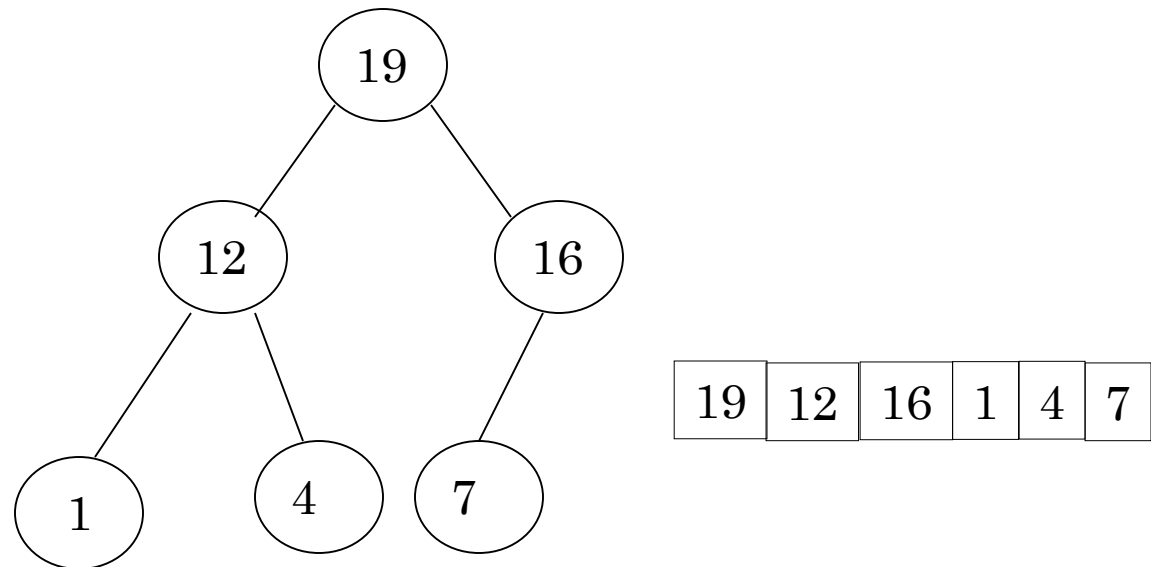
Heapify →



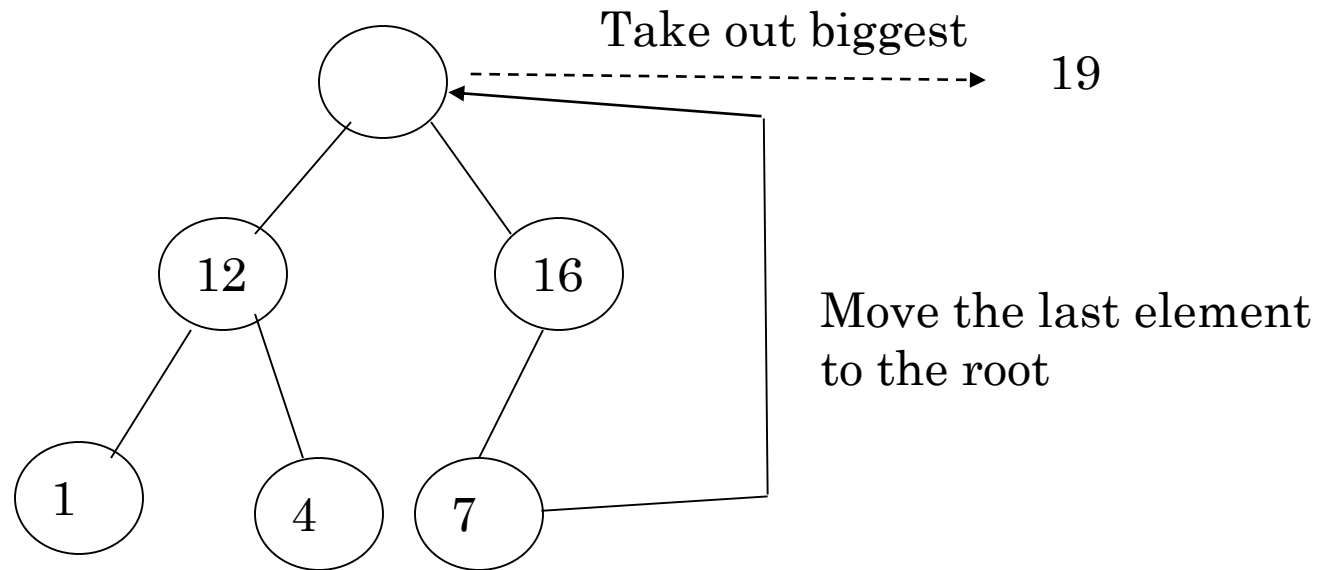
Max Heap



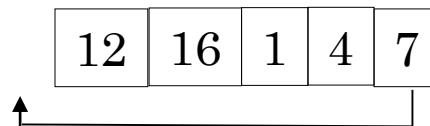
- To sort the elements in **increasing order**, use a **max heap**
- To sort the elements in **decreasing order**, use a **min heap**



DELETE ELEMENTS FROM MAX HEAP



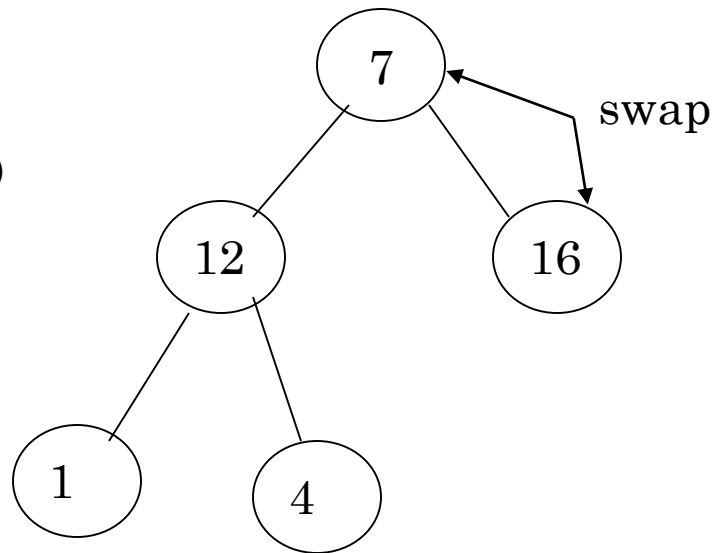
Array A



Sorted:



HEAPIFY()



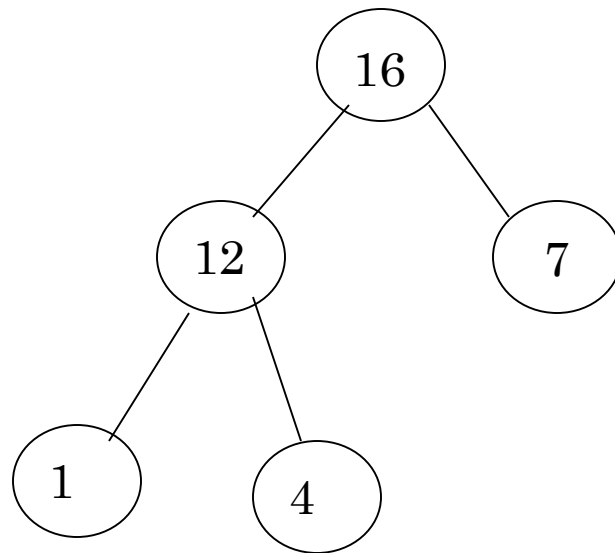
Array A

7	12	16	1	4
---	----	----	---	---

Sorted:

19





Array A

16	12	7	1	4
----	----	---	---	---

Sorted:

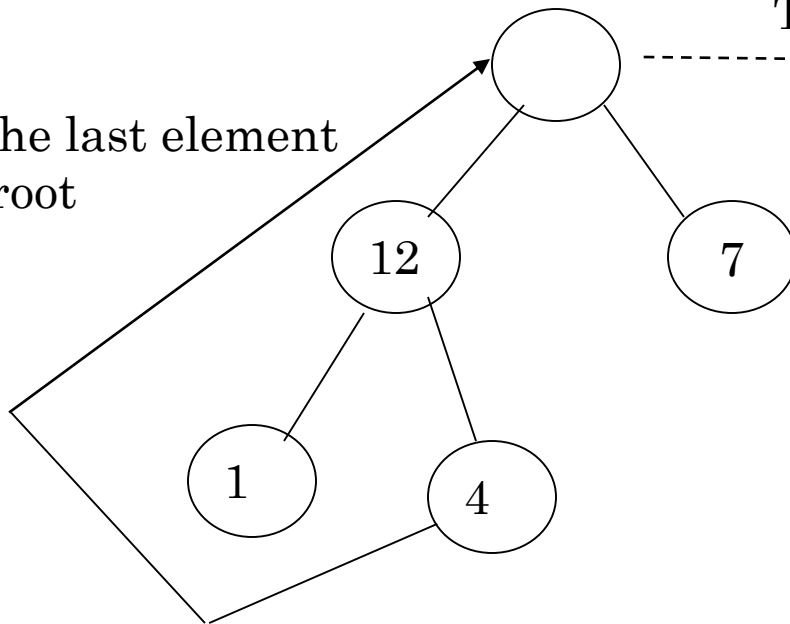
19



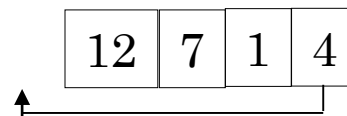
Take out biggest

16

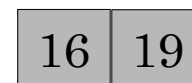
Move the last element
to the root

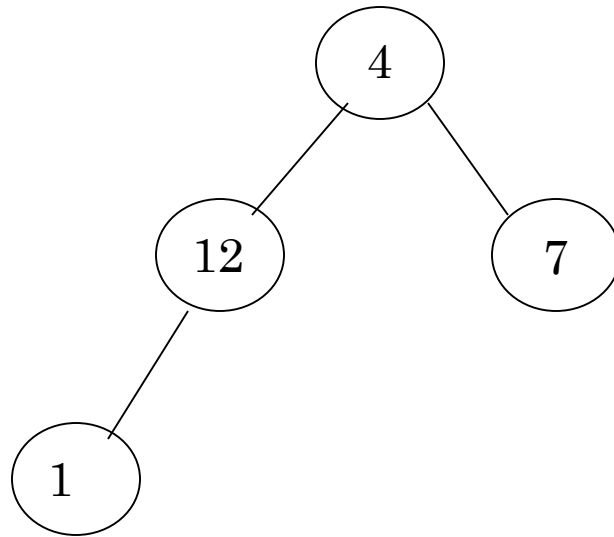


Array A



Sorted:





Array A

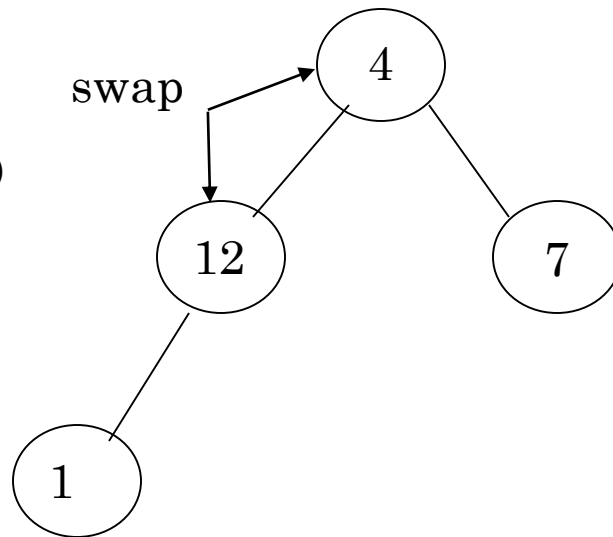
4	12	7	1
---	----	---	---

Sorted:

16	19
----	----



HEAPIFY()



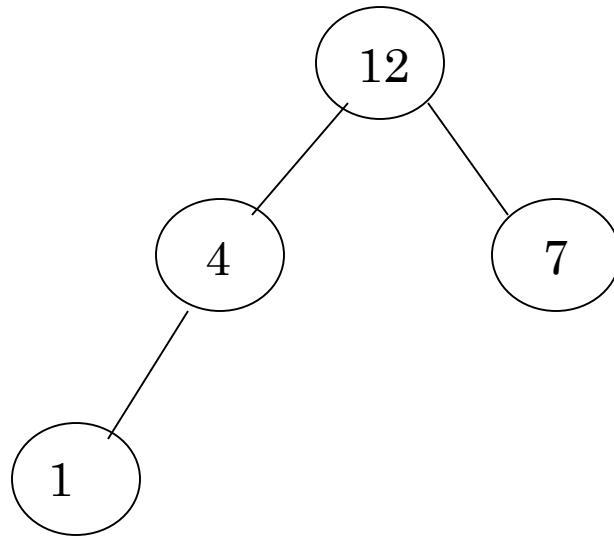
Array A

4	12	7	1
---	----	---	---

Sorted:

16	19
----	----





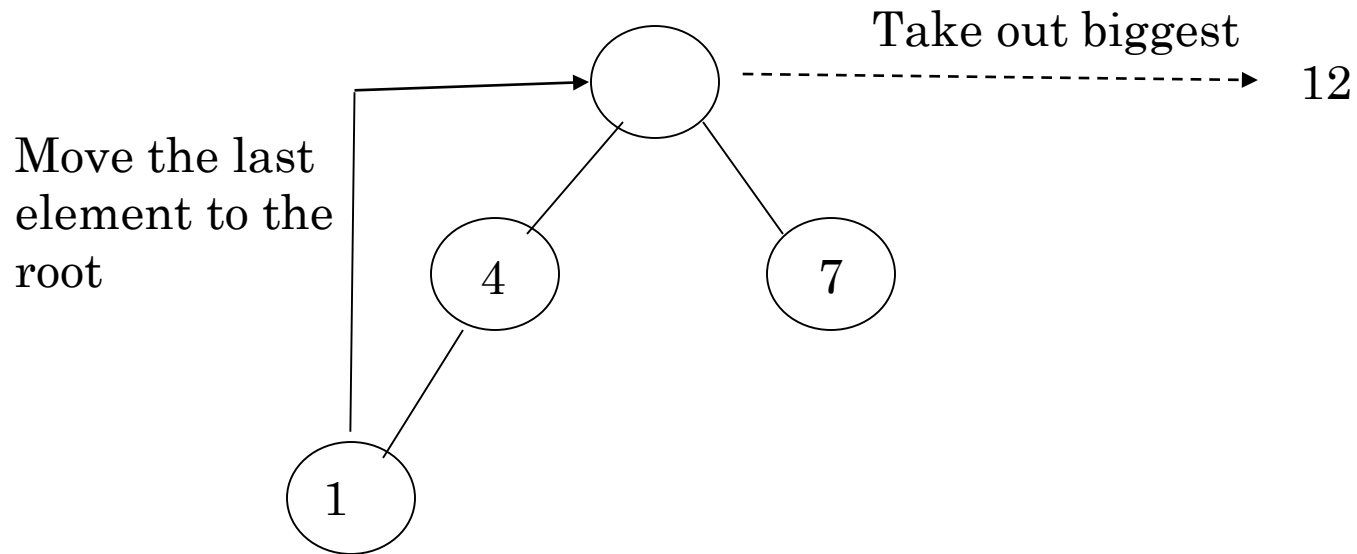
Array A

12	4	7	1
----	---	---	---

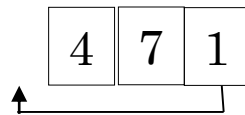
Sorted:

16	19
----	----

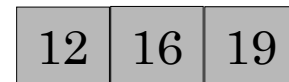


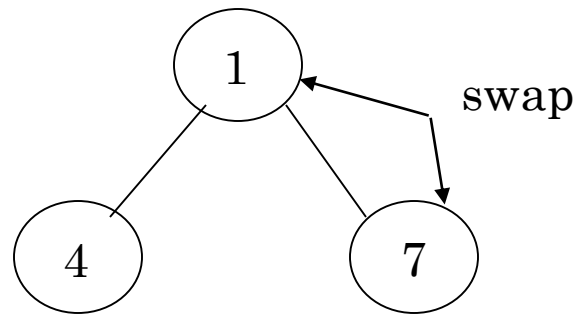


Array A



Sorted:





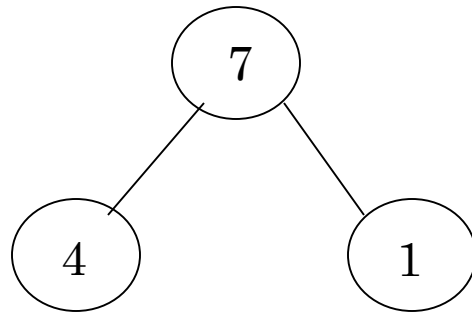
Array A

1	4	7
---	---	---

Sorted:

12	16	19
----	----	----





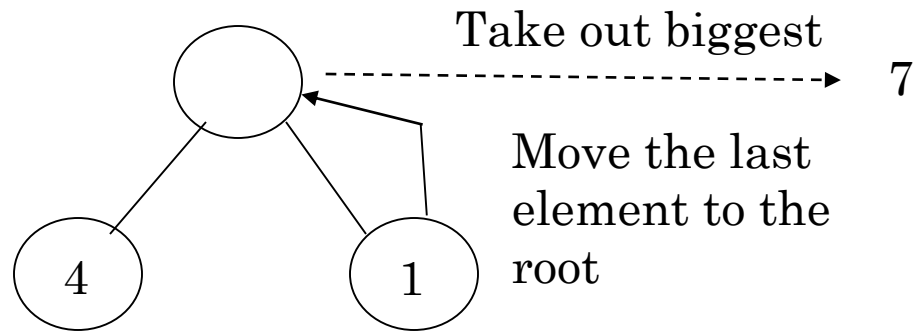
Array A

7	4	1
---	---	---

Sorted:

12	16	19
----	----	----





Array A

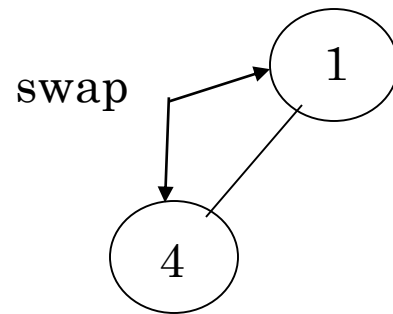
1	4
---	---

Sorted:

7	12	16	19
---	----	----	----



HEAPIFY()



Array A

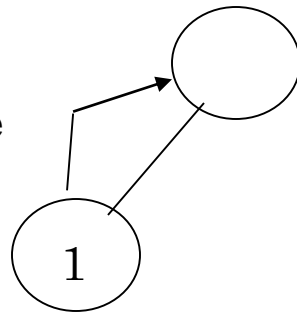
4	1
---	---

Sorted:

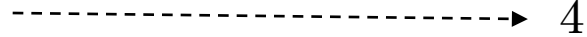
7	12	16	19
---	----	----	----



Move the last
element to the
root



Take out biggest



4

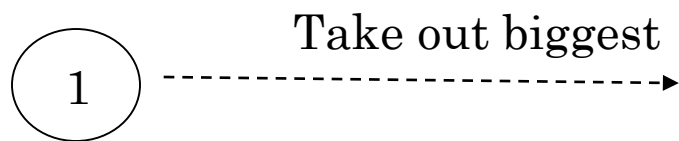
Array A

1

Sorted:

4	7	12	16	19
---	---	----	----	----





Array A

Sorted:

1	4	7	12	16	19
---	---	---	----	----	----



Sorted:

1	4	7	12	16	19
---	---	---	----	----	----

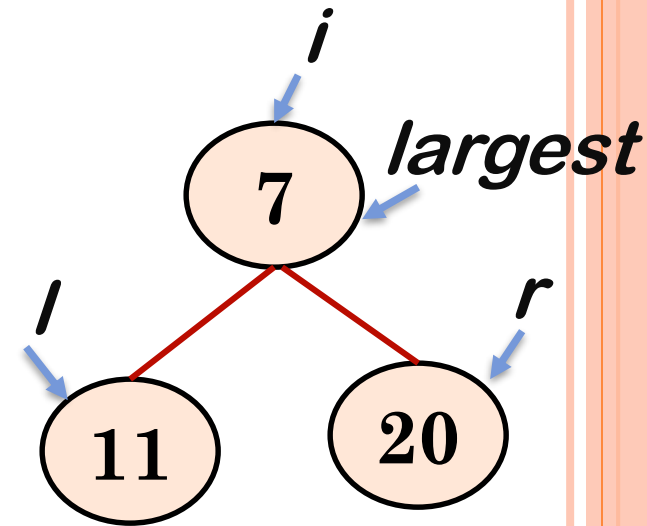


HEAPIFY

- Heapify picks the largest child key and compares it to the parent key. If the parent key is larger then heapify quits, otherwise it swaps the parent key with the largest child key. So that the parent now becomes larger than its children.

Heapify(A, n, i)

```
{  
    int largest=i;  
    int l ← left(2*i);  
    int r ← right(2*i)+1;  
    while(l ≤ n && A[l] > A[largest])  
    {  
        largest =l;  
    }  
    while(r ≤ n && A[r] > A[largest])  
    {  
        largest =r;  
    }  
    if (largest != i)  
    {  
        swap(A[largest],A[i])  
        Heapify(A, n, largest)  
    }  
}
```



Heapsort(A,n)

{

for(i =n/2,i>=1,i--) //Build Max heap

{

Heapify(A,n, i);

}

for(i =n,i>=1,i--) //Deleting elements from Max heap

{

swap(A[1],A[i]);

Heapify(A,n, 1)

}

}



Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

- The space complexity of the Heap sort is $O(1)$



POSSIBLE APPLICATIONS

- When we want to know the task that carry the highest priority given a large number of things to do
- Interval scheduling, when we have a list of certain tasks with start and finish times and we want to do as many tasks as possible
- Implementation of priority
- Sorting a list of elements that need an efficient sorting algorithm

